Frontier and Localhost: How Production AI Learns Outside the Weights

Mikhail L. Arbuzov
Independent Researcher
mike.arbuzov54@gmail.com

Lee Mosbacker
Independent Researcher
lee.mosbacker@gmail.com

Sisong Bei
Independent Researcher
qurining@gmail.com

Ziwei Dong
Independent Researcher
ziwei.dong@alumni.emory.edu

Dmitri Kalaev
Independent Researcher
kalaevdr@gmail.com

Alexey A. Shvets
Palo Alto Networks
ashvets@paloaltonetworks.com

Abstract

Production LLM systems increasingly adapt outside the model weights. After deployment failures, teams modify prompts, rules, memories, skills, tools, eval suites, routing graphs, and governance pipelines on a cadence that frontier weight updates cannot match. But this scaffold layer is still mostly maintained as *patchwork*: fixes are proposed by intuition, committed with weak credit assignment, accumulated without pruning, and promoted beyond the scope where they were validated. This paper formalises a disciplined alternative, which we call *artifact-layer descent*. The pipeline is concrete. Recurring residuals are assigned to scaffold coordinates. Candidate artifact deltas are tested against patch loss. Accepted deltas persist with rollback. Promotion across contexts is bounded by *evidence radius*: a local fix spreads only as far as the evidence supports. Surveying approximately 130 production and research systems from 2023 to 2026, we find current systems partially instantiate this loop but leave a central architecture gap, namely governed scaffold optimization across contexts. The contribution is not the claim that prompts are weights. It is to name the missing optimizer for an adaptation layer the field has already built.

1. The gradient moved outside the model

The standard mental model of an LLM system is the model. Pre-training fixes the weights, fine-tuning nudges them, deployment wraps a thin prompt-plus-tool layer around the result and ships. Production in 2025 and 2026 has stopped behaving this way. Frontier models update on cycles measured in months; the systems built on top of them update continuously, in markdown files, persistent memories, retrieved skills, registered tools, evaluation suites, agent topologies, version-controlled prompt repositories, PR-gated governance. A growing share of deployment-time adaptation now occurs outside the weights, in a fast-changing *local scaffold* wrapped around a slow-changing frontier. The section title is a metaphor. The gradient inside the model still happens during training, and the scaffold layer is not a gradient in the calculus sense. The substantive claim is the milder one. Production AI has accidentally created an external learning surface, and right now it is being operated as *patchwork* rather than as a disciplined optimization process.

Preprint.

Read together, these artifacts form a single external adaptation surface (the six substrates catalogued in §4) that current production practice still treats as scattered engineering. A Claude Skill is a learned procedural weight kept outside the model. A Cursor Rule is a local parameter for one project's distribution shift. A RAG connector is patch-specific memory access. A tool registry is capability provisioning along an axis the base model cannot reach. A PR review against a shared rule repository is a promotion gate on a federated update. An eval suite that blocks merge is a validation-loss check before promotion. A cross-tenant artifact repository is *federated artifact promotion* across distributed shards: scope expansion under a gate, not arithmetic averaging. None of these mechanisms is new; what is new is naming the surface and asking what optimization discipline it needs — and visibly lacks.

This paper surveys approximately 130 production and research systems, identifies the six substrates where the scaffold lives, formalises the two-loop architecture (local update, cross-context promotion under a gate), and names a missing architecture as the most consequential open problem: governed cross-tenant scaffold optimization with versioned lineage, role-based access, and rollback. The single-line thesis is this: **the field has built an external adaptation layer for LLMs, but not the optimizer that should govern it.** The rest of the paper develops that observation into a survey of where production sits on a *patchwork to random-search to descent* gradient, a formalisation of the disciplined limit case under stated assumptions, and a named missing architecture where the optimizer most plainly does not yet exist.

2. Why patch errors live outside the weights

The scaffold is not merely where production systems happen to store patches. For high-churn, tenant-specific, auditable adaptation, the scaffold is *often the practical substrate*. The reason is not that no alternative exists. It is that under deployment-time constraints (release cadence, tenant isolation, inspectability, rollback), the scaffold is cheap, locally scoped, reversible, and updatable at human time-scales while the weights are not.

Frontier training optimises for breadth. The weights are tuned for cross-patch generalisation. They smooth over local conventions, contradictory tenant preferences, and rare workflow-specific failures in order to preserve broad competence across millions of unseen deployments. The smoothing is the training objective, not an artefact. RLHF reduces output diversity and homogenises preferred styles across users [Kirk et al., 2024, Padmakumar and He, 2023], and instruction-tuning data composition rewards patterns that generalise across the annotator pool rather than patterns specific to any one deployment. The frontier model is therefore *trained to be unable* to encode a single tenant's idiosyncrasies as preferred behaviour. Tiwari et al. [2026] supply an independent two-timescale optimisation motivation from the in-context / in-weights side: in-context adaptation is cheap and rapid but capacity-bounded, in-weights adaptation is expressive but induces catastrophic forgetting and plasticity loss; the scaffold layer is the substrate that absorbs what in-context adaptation can carry cheaply, leaving the weights free to generalise.

Production reliability is achieved in depth. Inside a single deployment patch, reliability is determined by exactly what global training had to smooth away: local naming conventions, team-specific workflows, idiosyncratic tool chains, customer-specific risk tolerances, recurring local mistakes, hidden evaluator expectations. The per-patch quantities from our previous work are local by construction, and a library calibrated to one patch under-covers the next.

Encoding every patch in the weights is structurally infeasible. Three constraints rule out the obvious approach. *Economic:* millions of patches multiplied by billions of parameters times per-patch fine-tuning cost is not a viable training-compute budget. *Release-cycle:* the patch wants daily updates; weight releases ship quarterly at best. *Cross-patch interference:* overfitting the weights to one tenant's preferences degrades adjacent tenants whose preferences contradict. The result is brittle in exactly the way the scaffold is not.

Per-tenant fine-tunes, LoRA adapters, distilled small models, and learned routing each absorb some patch residuals in principle. In practice the scaffold dominates, because it is the substrate that satisfies all four production constraints at once: cheap, inspectable, reversible, locally scoped, and updatable on a daily cadence rather than a release cadence. The placement question that *Architecture of Errors* left open (where do the interventions live?) has this as its answer. The frontier model generalises. The local scaffold specialises. Reliability comes from governing that specialisation.

Three stages of scaffold maintenance recur across the corpus. They differ in how candidate updates are produced, whether credit assignment is explicit, and whether promotion is evidence-bounded. Most production deployments sit at Stage 1 or 2. The math of §3 applies to Stage 3.

Stage	Update mechanism	Credit assignment	Promotion discipline	Dominant failure mode
1. Patchwork	Intuition-driven local edits	Implicit / ad hoc	None	Bloat
2. Random artifact search	Many variants tried under eval	Weak; eval is the only signal	Eval-gated CI but no failure-to-coordinate tracing	Overfitting to in-distribution patch
3. Artifact-layer descent	Residual to coordinate to delta	Explicit per failure mode	Evidence-radius-bounded; reversible	(the discipline this paper formalises)

The paper's central claim is not that today's production systems already run this loop. Most do not. The claim is that a patchwork loop *becomes* artifact-layer descent once the five conditions of §3 are in place. The survey of §§5–8 asks which deployed systems have closed which of them; §9 names the one closure no public system has yet reached.

3. Artifact-layer descent

The disciplined version of scaffold maintenance is one loop, run again and again over the life of a deployment. It does not react to single failures; it works from their accumulated record, and it ends with a decision about how widely to trust each fix. As the deployment runs, it logs what went wrong — failed outputs, user corrections, rejected answers, eval regressions, tool-call errors. Reflecting over that record, the loop clusters the failures into recurring *failure modes* and takes up the ones that matter most: a mode earns attention in proportion to how often it recurs and how badly it fails — repeatability times severity, its contribution to the deployment's failure rate. For such a mode, locate the single scaffold coordinate responsible — a stale memory entry, an under-specified instruction, a missing tool argument, a brittle retrieval rule, a misrouted agent edge — and synthesise one candidate edit to it. Measure whether the edit lowers *patch loss* — the deployment's failure rate on its own recurring tasks — and let a gate accept or reject it on that measurement. Keep an accepted edit in the scaffold; prune it or roll it back if a later check shows it made things worse. Then take the step most systems skip: decide how far the edit should travel, and promote it to other deployments only as far as it has actually been validated. That last limit — a fix earned on one deployment spreads only as far as its evidence reaches — is the edit's *evidence radius*. The rest of this section makes each step precise; §§4–9 then ask which deployed systems have closed which of them.

Most current production does not close this loop. A failure happens, someone adds a prompt or rule or memory entry, the artifact pile grows, nobody knows what caused improvement, nobody knows when to prune, and fixes leak into broader contexts. This is patchwork (Stage 1 of §2), not stochastic descent. Even systems with eval-gated CI (Stage 2) usually lack failure-to-coordinate credit assignment, so improvements are real but unattributed and brittle to transfer. Scaffold updates *become* stochastic descent (Stage 3) only when five pieces are in place: a measurable patch loss; captured residuals; a failure traced back to the scaffold coordinate that caused it; a synthesised candidate edit; and a gate that checks whether the edit lowers loss before it is kept or promoted. The descent picture below is mechanism when those five conditions hold, and modeling language otherwise.

Two loops, in plain English first. *Loop 1* is the local fix: a recurring failure type produces a candidate edit, the edit is tested, and the accepted edit changes one deployment's scaffold. *Loop 2* is the separate decision of whether that local fix should be shared with other deployments — promoting it from one repository to a team-wide template, from one tenant's playbook to a cross-tenant skill library, and so on. The two loops turn different knobs. Loop 1 improves the scaffold against a single deployment's own failures; Loop 2 changes *which* deployment's failures the scaffold is being tuned against.

Assumptions and scope of the formalization. The descent picture is only as good as the conditions it rests on, so we state them before formalising it; where a condition fails, the same picture should be read

as analogy rather than mechanism. The argument below holds when: (i) the frontier model's weights stay frozen while the scaffold is being adapted; (ii) each scaffold edit is a persistent, inspectable artifact, not a transient prompt that vanishes after one session; (iii) the deployment exposes a loss you can actually measure — a failure rate, a correction rate, an eval score; (iv) the process that proposes edits can, at least sometimes, trace a failure back to the scaffold coordinate responsible for it; (v) the gate is a noisy estimator of whether an edit helps, not an oracle that knows the true change in loss; and (vi) Loop-2 promotion widens an artifact's scope under a gate, rather than numerically averaging comparable updates the way weight-space federated learning does. Survey claims of the form "no surveyed system has X" are bounded by the audited public corpus and the survey cutoff of May 2026.

Patch scope hierarchy. *Patch* is used throughout this paper with a defined scope hierarchy: USER $\subset$ PROJECT $\subset$ STACK $\subset$ TENANT $\subset$ CORE. Throughout, *patch* denotes a deployment scope, never an individual fix; the undisciplined practice of editing the scaffold by hand is *patchwork*. A USER patch is one human's account or fork. A PROJECT is one repository, workflow, or product workspace. A STACK is a curated bundle of projects under one team. A TENANT is an organisation or customer. CORE is the cross-tenant universal scope. Each level has its own loss — the average failure rate over all the patches it contains. When the text says "cross-tenant promotion" it specifically means moving artifacts up to the TENANT or CORE level. "Patch-specific adaptation" without qualifier means descent on a USER or PROJECT-level loss.

The patch loss is the expected residual failure rate of the model-scaffold composite on patch D:

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_s is a structured scaffold parameter object over six substrate coordinates and ℓ is an operational loss (failure indicator, user-correction rate, eval error, schema violation, reviewer disagreement). The important distinction is substrate. θ_M is fixed during deployment. θ_s is the object being fit.

A failure does not yield an analytic gradient. It yields a candidate edit — synthesised by reflection over the failure history, and proposed in order of the recurrence $\times$ severity of the mode it addresses. The gate G_t accepts the edit if its noisy estimate of the directional loss change clears a margin. The accepted edit applies to the current scaffold; rejected edits leave the scaffold unchanged:

$$\theta_s^{t+1} = \begin{cases} \theta_s^t \oplus z_t, & G_t(z_t) = 1, \\ \theta_s^t, & G_t(z_t) = 0, \end{cases}$$

with the gate accept condition

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\,\Omega(z_t) \le -\tau_t,$$

where $\widehat{\Delta}$ is the gate's estimator of the loss change, $\Omega \ge 0$ is a complexity penalty on the edit (instruction length, tool-permission breadth, promotion scope), $\lambda \ge 0$ regularises that penalty, and $\tau_t \ge 0$ is the required improvement margin. Production systems implement this without writing the equation: an eval blocks a merge on regression, a reviewer rejects a vague rule, a governance system prevents a local artifact from being shared until evidence accumulates. Different surveyed gate families instantiate the abstraction differently. Eval-cascade, RL-threshold, LLM-judge, surrogate verifier, and reviewer-judgment families directly populate the per-edit gate. Semantic-merge and MAP-Elites archive selection act as proposal-pruning steps that *precede* the per-edit gate. The per-system mapping is in Appendix C.

Two loops, two distinct knobs. Consider a concrete case. An engineer notices that a coding agent keeps misnaming a class in one repository. The fix is a one-line rule in that repo's instruction file. The edit is small. Its scope is one project. The same engineer could promote the rule to a team-wide template, or to the company's CORE instructions, or to a cross-tenant skill registry. The *edit* did not change. The *scope* did. And the gate that should let it through gets stricter at each step, because the loss it is being tested against is now an average over more patches, and a fix that helps one repo can hurt another.

That observation maps onto two distinct knobs in the optimisation analogy. The first is *edit magnitude* — how much a single accepted edit changes the local scaffold. This is the analogue of a learning rate:

it sets how big each step is. The second is *promotion radius* — how many downstream patches the accepted edit is applied to. This does not change the step size; it changes the *target* the step is aimed at, that is, which average failure rate the scaffold is being tuned to reduce. Accept an edit at the USER level and you are reducing the failure rate on one patch (L_D). Promote it to a PROJECT and you are now reducing the average failure rate across all of that project's patches; promote it to TENANT or CORE and the target becomes a federated average over many tenants' patches. A larger radius therefore aims the same edit at a broader, harder-to-please target, and an edit that lowers the narrow average can raise the broad one. The two knobs are set independently, and the gate must be calibrated against both. This is why governance is not administrative overhead. It is the regularization system that keeps a small fix from being aimed at the wrong target.

Loop 2 is *federated artifact promotion* — selection, merge, curation, review, distribution — not arithmetic averaging of numeric updates; §9 sharpens the FedAvg analogy and its limits.

Each loop has a gate. Gates are **Auto-gated** when the decision is an operationalised algorithmic criterion (RL threshold, eval score, LLM-judge with quantified agreement, surrogate verifier, benchmark cascade). They are **Human-gated** when the decision is reviewer judgment. They are **None** when the loop is absent. These five terms (Loop 1, Loop 2, Auto-gated, Human-gated, None) anchor §§5–9.

The descent argument is *mechanism* when these five conditions hold and *metaphor* when they are absent. Appendix E formalises the proof assumptions A1–A5 — distinct from the operating assumptions (i)–(vi) above — needed for a one-step descent inequality and a convergence proposition of the loss to a G-stable scaffold, and works through the RAG, pitch-review, tool-use, and memory cases that make the mechanism visible.

3.5. SkillOpt and the convergence on artifact-layer descent

Recent work has begun to operationalise all five §3 conditions for skill-document adaptation, and SkillOpt [Yang et al., 2026] is the clearest case: it treats a single markdown skill document as the trainable external state of a frozen agent and edits it with seven explicit deep-learning-style controls — rollout batches; reflection minibatches; add/delete/replace edits; a textual learning-rate budget with cosine decay; a held-out selection-split accept gate; a rejected-edit buffer that keeps failed proposals as negative feedback; and an epoch-wise slow/meta update for longer-horizon consolidation. Its framing — *skill optimization as deep-learning-style optimization over an external natural-language state* — is exactly the disciplined gradient-like update mechanism §3 argues is missing in current production. SkillOpt is not an isolated proof of concept but the cleanest member of a rapidly crystallising space: the same convergence pressure this paper identifies is arriving from at least 14 independent directions in 2025–2026, each instantiating a different subset of §3's five conditions and targeting a different substrate (skill document, harness code, system prompt, shared memory).

Mapped onto §3, SkillOpt instantiates each abstract quantity with a concrete mechanism; Appendix G expands every row with the exact algorithm-level mechanism, including pointers to SkillOpt's Algorithm 1 line numbers.

§3 quantity	SkillOpt instantiation	Mechanism	Numerical signature
θ_s (scaffold)	`best_skill.md`	persistent text artifact, exported as the deployment object	379–1,995 tokens; $\times$2.5–53 growth
L_D (patch loss)	held-out hard / exact-match score	per-benchmark evaluator	6 benchmarks
Q_t (proposal)	optimizer-model reflection over failure and success minibatches	LLM-judge over rollout batch	B_m=8, parallel reflectors
z_t (edit)	add/delete/replace patch on the skill	structured JSON edit	1–4 accepted edits per skill
L_t (textual learning rate)	bounded edit budget per step	constant/linear/cosine/auto schedule	$L_t \in \{1, 2, 4, 8, 16\}$
G_t (gate)	held-out selection-split accept gate	strictly-greater validation	ties rejected

§3 quantity	SkillOpt instantiation	Mechanism	Numerical signature
rejected-edit buffer	epoch-local store of failed proposals	training-time negative feedback only	zero inference cost
slow/meta update	epoch-wise longitudinal guidance written to a protected slow-update field	second-order consolidation	-22.5 pts on SpreadsheetBench when removed

The headline empirics make the descent picture testable rather than aspirational. Across six benchmarks, seven target models, and three execution harnesses (direct chat, Codex, Claude Code), SkillOpt is best or tied-best on all 52 evaluated (model, benchmark, harness) cells, lifting GPT-5.5 by +23.5 / +24.8 / +19.1 points over no-skill under direct chat / Codex / Claude Code, and beating the strongest per-cell baseline drawn from TextGrad, GEPA, Trace2Skill, EvoSkill, human-skill, and one-shot-LLM by +5.4 points on average. Transfer is also measured: a SpreadsheetBench skill trained inside the Codex loop transfers to Claude Code at +59.7 over that harness's no-skill baseline, and an OlympiadBench skill yields positive gains on Omni-MATH at every model scale tested. In §3 vocabulary: SkillOpt closes Loop 1 under an algorithmic gate, runs an epoch-wise auto-gated stabiliser as a second algorithmic loop, and reports cross-model / cross-harness / cross-benchmark transfers that read as *empirical evidence-radius measurements* — Loop-2 scope expansion that is sound for the demonstrated radii while no cross-organisational governance is shipped.

The convergent landscape. Five clusters of concurrent work instantiate complementary subsets of §3's mechanism stack, each targeting a different substrate or exposing a different constraint: *reflective-evolution* [GEPA, Agrawal et al., 2025; Tiwari et al., 2026], *reflective context* [Vassilyev et al., 2026], *harness optimisation* [Meta-Harness, Lee et al., 2026; AHE, Lin et al., 2026; MOSS, Cai et al., 2026; CANTANTE, Zehle, 2026; Ong et al., 2026], *multi-objective + lifecycle* [MOCHA, Tanjim et al., 2026; Ratchet, Zhang et al., 2026b], and *cross-task transfer* [ICT, Shalev et al., 2026; Combee, Li et al., 2026; CORAL, Qu et al., 2026; PACE, Ling et al., 2026]. The table below indexes what each instantiates in §3 and what it leaves open; Appendix G walks each cluster in detail with its §3 mapping stated explicitly.

Cluster	Representative systems	§3 mechanism instantiated	What is left open
Reflective-evolution	GEPA; Tiwari et al.	Q_t over trajectory evidence; two-timescale motivation	persistent θ_s; L_t budget; held-out G_t
Reflective context	RCL (Vassilyev et al.)	θ_s over context space; $\Omega(z)$ as generalisation penalty	cross-deployment Loop 2
Harness optimisation	Meta-Harness; AHE; MOSS; CANTANTE; Ong et al.	credit assignment over S5; source-level extension	governed Loop 2 across orgs
Multi-objective + lifecycle	MOCHA; Ratchet	constraint Pareto frontier; hygiene-as-precondition	gradient-analogue closure
Cross-task transfer	ICT; Combee; CORAL; PACE	empirical evidence-radius; async multi-agent Loop 2 within one org	cross-organisational gate, lineage, rollback

What the convergence establishes. Read against §3's mechanism stack and §9's four-property criterion, the family establishes four things: the scaffold-versus-weights decomposition is independently confirmed; credit assignment is the central engineering bottleneck; lifecycle hygiene is a precondition for descent rather than an alternative to it; and no member combines Loop-1 automation with cross-organisational Loop-2 governance — so §9's gap is confirmed, not reduced, by the convergence.

Appendix G develops each claim, gives the full SkillOpt correspondence and its contrast against the six measured baselines, and walks each convergent cluster.

4. Six scaffold substrates

A patch-local scaffold composes six substrates. They are the coordinates of θ_s, each independently editable.

Substrate	What persists	Score-5 mark
S1. Instructions	Project/team/org rules attached to a workspace	Versioned, two-loop, Auto-gated promotion
S2. Skills	Procedural artifacts (code/text) loaded on demand	Skill bank self-modifies on failure, unit-test gated, versioned, promoted
S3. Memory	Cross-session experience (episodic / semantic / procedural)	Provenance + versioning + correction pathways
S4. Tools	Vetted action surface + context schemas	Domain-curated bundle is the product; eval-gated tool versions
S5. Orchestration	Multi-role graph + routing	Population/policy updates from outer-loop signal
S6. Governance	Versioning, promotion, audit, rollback	dev→staging→prod with eval thresholds + audit log + rollback

The 0–5 rubric is uniform across substrates: **0** ephemeral; **1** persistent local; **2** persistent + tool attachment, no feedback; **3** multi-scope or feedback but no automated promotion; **4** automated feedback updates the scaffold (one loop closed); **5** two-loop versioned promotion plus governance (both loops closed). We adopt this six-coordinate decomposition because each coordinate is independently editable in production systems; alternative partitions (e.g., separating evals from governance) are possible, and the descent argument of §3 is invariant to the partition so long as the independent-editability property holds.

5. Survey method

Scope and corpora. The survey covers approximately 130 unique LLM systems published or productionised between January 2023 and May 2026. The candidate pool was assembled by searching the public discourse for systems described as "self-improving," "scaffolded," or "agentic," then filtering by whether the system wraps a foundation model with at least one persistent artifact updated from deployment signal. Approximately 90 systems satisfy that *patch-local discriminator* (score ≥ 3 on at least one of the six substrates) and form the **core corpus**. The remaining ~38 systems carry the name but fail the discriminator; we keep them as a **contrast corpus** so the boundary is auditable rather than rhetorical (Appendix D lists representative entries).

Discriminator and composite criterion. The score ≥ 3 cut admits a system as patch-locally adaptive. A separate, stricter audit applied to 22 candidate systems uses a *composite two-loop* criterion: Loop 1 modifies a persisted artifact from session signal, *and* Loop 2 has an explicit cross-context promotion mechanism with versioning or lineage. Fourteen research systems pass the strict audit. Two industry exemplars (Sierra Agent OS 2.0, Cognition Devin) are also evaluated using vendor documentation as primary evidence; they are flagged in the §8 matrix because the evidence tier is weaker. The strict count moves under two natural relaxations of the criterion (counting versioning OR lineage OR rollback as governance; admitting optimiser-state updates as Loop-1 persistence), but the qualitative findings of §§6–9 survive both. Appendix B gives the derivation.

Evidence policy. Each system carries one or more of: (T1) peer-reviewed publication; (T2) arXiv preprint; (T3) production claim with metrics; (T4) open-source artifact; (T5) industry-blog or interview-level documentation. T1 and T4 carry the highest weight in cluster analysis. T3 is

admissible when the claim is concrete (specific benchmark, named integration target). T5 is admissible only when corroborated by a higher tier elsewhere. Tier flags propagate to cluster claims through the per-row evidence column in Appendix A; production claims are not pooled with peer-reviewed evidence when computing cluster-level statistics.

6. Substrate maturity is uneven

Aggregating the core corpus by substrate gives the picture below. The column n counts surveyed systems on that substrate, *Mean* is the average score, and *Score-5 systems* are the few that close both an automated inner loop and a governed two-loop promotion at the substrate level.

Substrate	n	Mean	Score-5 systems	Survey finding
S1. Instructions	12	2.83	*(none)*	Rules persist across IDEs and agents (Claude Code's CLAUDE.md, Cursor Rules, AGENTS.md, Windsurf, Continue.dev) but no surveyed instruction system closes both an automated inner loop and a governed two-loop promotion. Ceiling held at 4 by Claude Code and Replit
S2. Skills	16	2.62	SAGE, COSPLAY	Research frontier is active (failure-triggered skill update); production governance is weaker. Anthropic's skills repository is one-pool with PR review but no automated eval gate
S3. Memory	19	3.16	Cuadros et al. governed collaborative memory	Cross-session memory is widespread; *correctable, versioned* memory is rare. Production memory systems (ChatGPT Memory, Claude Memory, Mem0, Zep, MemGPT) treat memory as append-only
S4. Tools	19	3.63	MCP, Harvey, Hippocratic AI	Most mature production substrate. Tool bundles are the commercial unit; MCP is the cross-vendor standard with broad public-server ecosystem and substantial enterprise adoption
S5. Orchestration	19	2.68	*(none)*	Multi-agent topologies exist (LangGraph, AutoGen, CrewAI) but topology *evolution* is rare; MAE comes closest with RL co-evolution of Proposer/Solver/Judge population but lacks Loop-2 cross-context promotion
S6. Governance	21	3.05	Braintrust, Vellum, LangSmith Hub, AGENTS.md/AAIF	High production maturity for prompts-as-code: immutable commits, eval-gated PR merge, sub-five-minute rollback, typically without inner-loop adaptation
INTEGRATED	34	3.74	NanoResearch, AutoAgent, SkillRL	The high-water mark when present, but the cluster is dominated by research systems lacking enterprise governance

Two things stand out. First, the field has independently converged on the six substrates: every mature system has a recognizable instance of each, and the names differ across vendors but the role does not. Second, no system matures all six at once. Research systems learn fast and lack enterprise governance. Production systems govern well and learn slowly. Open standards solve artifact portability but not adaptation. Vertical-bundle vendors solve patch specificity but rarely expose cross-tenant promotion. The maturity gradient is not a quirk of the corpus; it tracks which constraints each kind of system was built to satisfy.

Five absences fall out of the discriminator and recur as Paper-4 targets in §11. Failure-triggered memory is rare in production (most deployed memory systems record preferences and successful facts, not residuals). Cross-customer skill promotion is absent (Anthropic's skills repository is one-pool; AGENTS.md is cross-vendor but not cross-customer). Memory versioning is mostly unsolved outside the Cuadros et al. framework. Scaffold-level curriculum (which sessions inform the gradient) is

unspecified everywhere; every session contributes equally. Held-out evals at Loop-2 promotion are absent, which is the missing piece that would catch a scaffold over-fitting one deployment before it propagates to another.

7. Four clusters

Four archetypal clusters recur across the core corpus. They differ in where on the *patchwork to descent* gradient each cluster sits and in which loop is automated. This section cuts the corpus by architectural archetype; §8 re-cuts the composite-two-loop subset by gate type.

Cluster	Representative systems (year, PP score)	Loop 1 / Loop 2	What the cluster proves
Research full-auto	NanoResearch (2026, 5), SkillOpt (2026, 5), SkillRL (2026, 5), AlphaEvolve (2025, 4), DGM (2025, 4), ADAS (2024, 4), Voyager (2023, 4)	Auto / Auto	Both loops can be fully automated under algorithmic gates; benchmark gains compound; enterprise governance is uniformly absent
Production hybrid	SkillForge (2026, 4), CASCADE (2025, 4.5), Sierra OS 2.0 (2025, 3)[1], Cognition Devin (2024, 3)	Auto / Human	Algorithmic Loop-1 gate plus reviewer-judgment Loop-2 gate is the production-viable cell; SkillForge is the architectural exemplar
Governance-first	Braintrust, Vellum, LangSmith Hub (all 2024, S6 = 5)	None / Auto	Eval-gated promotion exists; candidate generation is manual; no patch-local inner loop
Open standards	AGENTS.md (2025), MCP (2024), Claude Skills (2025), Cursor Rules (2024)	Mixed / Mixed	Vendor-cross convergence on artifact format and tool attachment; tens of thousands of AGENTS.md repositories; broad MCP server ecosystem with substantial enterprise adoption

Research full-auto systems demonstrate that the architecture works under fully algorithmic gates. NanoResearch co-evolves Skills, Memory, and Policy with a semantic-merge orchestrator. SkillOpt operationalises the deep-learning-optimizer analogy explicitly, with held-out validation gating, rejected-edit buffers, cosine-decay textual learning rate, and epoch-wise slow/meta consolidation that together resemble standard SGD machinery applied to a markdown artifact (§3.5; Appendix G). SkillRL uses RL reward distillation with a hard threshold at task-category accuracy below 0.4. AlphaEvolve runs a Gemini-driven ensemble with a cascade evaluator and a MAP-Elites archive, deployed at Google to gain +0.7% Borg compute worldwide, +23% on the Gemini training kernel, +32.5% on FlashAttention, and the first general-field improvement over Strassen's bound on 4×4 matrix multiplication. DGM modifies its own source code with archive-based parent selection and

[1] Sierra Agent OS 2.0 also fits the *open standards / vertical-bundle* discussion below because its commercial product wraps a vertical workflow around the standardised agent-OS surface; we place it primarily in *Production hybrid* on the basis of its Loop-1 and Loop-2 gate types.

lifts SWE-bench from 20% to 50%. The cluster maximises velocity. Enterprise governance is uniformly absent.

Production hybrid systems pair an algorithmic Loop-1 gate with a reviewer-judgment Loop-2 gate. SkillForge drives auto-generated skill diffs through an LLM-judge with >90% human agreement (Loop 1), then routes what the judge passes through human support engineers (Loop 2), with VFS version commits providing lineage. CASCADE reports the largest within-benchmark ablation gain in the corpus (+57.9 pp on SciSkillBench), driven by memory consolidation gated by human-agent collaboration; the headline gain is not comparable across benchmarks because SciSkillBench was designed to reward the very mechanism CASCADE adds. Sierra Agent OS 2.0 routes AI-driven Insights through human-authored Expert Answers in GitHub-style Workspaces. Cognition Devin ships dynamic in-session tool creation with a 67% PR-merge rate, where the merged PRs are the Loop-2 promotion. This is the production-viable cell.

Governance-first systems automate Loop 2 alone. Braintrust, Vellum, LangSmith Hub, W&B Weave, Agenta, and LangFuse all instantiate the *prompts-as-code* pattern: immutable commits, semantic-version tag pointers, PR-blocked merges on eval regression, sub-five-minute rollback. Loop 1 is absent because the candidate artifacts are engineer-authored at their desks rather than triggered by session feedback. We frame this as *MLOps for scaffolds*, distinct from MLOps for weights.

Open standards systems converge across vendors on the artifact format and the tool layer. AGENTS.md is now in over 60,000 GitHub repositories with measured efficiency gains: Lulla et al. [2026] report roughly -29% wall-clock and -17% tokens with no quality loss. MCP has become the cross-vendor tool standard with thousands of public servers and substantial enterprise adoption. Claude Agent Skills, Cursor Rules, GitHub Copilot custom instructions, Windsurf rules, Continue.dev rules, Zed AI rules, and Replit `replit.md` all converge on a git-tracked markdown artifact attached to a workspace, with an MCP-style tool layer for actions. Convergence across vendors is the strongest evidence the architecture is real rather than any one vendor's local optimum.

The four clusters do not partition the corpus (many systems sit between two clusters), but they capture the architectural variation. Vertical-bundle vendors (Harvey for legal, Hippocratic AI for healthcare) sit between *open standards* and *production hybrid*: the bundle is the commercial unit; the underlying model is mostly the same one a competitor would use. One result from the orchestration literature also constrains the picture. Tran and Kiela [2026] report that single-agent LLMs match or beat multi-agent systems on multi-hop reasoning when thinking-token budgets are matched. The implication is that observed maturity gaps on S5 may reflect the rarity of governed topology *evolution*, and the conflation of agent-count with compute-budget, rather than a structural deficit of multi-agent designs.

8. The two-loop design space

The composite-two-loop systems sit in a 3×3 matrix indexed by gate type on each loop.

	Loop-2 Auto-gated	**Loop-2 Human-gated**	**Loop-2 None**
Loop-1 Auto-gated	NanoResearch, SkillOpt, SkillRL, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoAgent, AutoManual	SkillForge, CASCADE, Sierra OS 2.0†, Cognition Devin†	MAE*, MetaGen*, MetaReflection*, CLIN*
Loop-1 Human-gated	**[empty by engineering logic]**	**[empty in surveyed corpus]**	*(none)*
Loop-1 None	ExpeL (weak), Braintrust*, LangSmith Hub*, Vellum*	Anthropic skills repo*, DSPy*, TextGrad*	Self-Refine*, ReAct*, Mem0*

Asterisks fail at least one composite-two-loop criterion; included for contrast. † Industry exemplar evaluated using vendor documentation as primary evidence.

Three cells are populated. **[Auto × Auto]** holds twelve research systems with machine-evaluated decision criteria on both loops. **[Auto × Human]** is the production-viable cell: Loop 1 fires at machine speed under an algorithmic criterion, Loop 2 requires human approval before propagation. **[None × Auto]** holds the governance-first prompts-as-code systems with eval-gated CI but no failure-triggered candidate generation. Three cells are empty for distinct structural reasons. **[Human × Auto]** is empty by engineering logic: if humans author candidates manually, automated promotion offers little leverage. **[Human × Human]** is empty in the surveyed corpus; a candidate occupant would be a solo or small-team workflow where the marginal cost of a bad auto-merge exceeds the marginal cost of human latency, and no public example was found in the May 2026 window. **[Human × None]** is structurally improbable: a system requiring human authorship on Loop 1 without any Loop 2 collapses to engineer-edits-a-config, and is not patch-local in the sense of §3.

Two cross-cutting findings sharpen the picture: noise reduction via batching is largely absent (every auto-gated system fires at $n = 1$ per generation step, with SkillOpt a partial exception), and multi-layer hierarchies are under-explored (only AlphaEvolve shows explicit depth). The full cell-by-cell discussion, these two findings in detail, the closest-approaches enumeration that supports §9, and the DGM dual-mode footnote are in Appendix F.

9. The missing architecture: governed cross-tenant scaffold optimization

Four properties define the architecture that no surveyed system fully instantiates:

- (a) **Auto-gated Loop 1.** The commit criterion is an algorithmic threshold.
- (b) **Multi-tenant cross-org promotion.** Loop 2 routes artifacts between organisations.
- (c) **Versioned lineage with RBAC.** Promoted artifacts carry semantic versions, audit trails, and access-control policies.
- (d) **Rollback.** A bad promotion can be reverted without redeploy.

The closest approaches partition the requirements. AlphaEvolve has (a) and partial (b) within a single organisation. SkillForge has (a) and (b) within one enterprise with partial (c) via VFS commits. SkillOpt has (a) fully and partial (c) via the exported `best_skill.md` artifact and the protected slow-update field, and reports positive cross-model / cross-harness / cross-benchmark transfer as empirical evidence-radius measurement; it does not implement (b) cross-organisational promotion or (d) explicit rollback. CORAL [Qu et al., 2026] has partial (a) via heartbeat-gated promotion and asynchronous multi-agent Loop 2 within one system; no (b) cross-organisational scope, no formal (c) RBAC, no (d) rollback. PACE [Ling et al., 2026] has (a) at the small-model production timescale and a fast/slow two-timescale Loop-1 + Loop-2 that consolidates across one deployment's own episodes; no cross-deployment (b), no formal (c) or (d). Meta-Harness [Lee et al., 2026] has (a) over the harness substrate (S5) with a filesystem-of-candidates credit-assignment record; no Loop-2 cross-organisational promotion, no RBAC or rollback. Sierra Agent OS 2.0 has (b) and partial (a). The governance-first cluster has (c) and (d) explicitly but no (a) or (b) in the strict sense. The convergent evidence sharpens rather than reduces the §9 gap: every closest approach satisfies a non-trivial subset of (a) and (c), but no member yet provides (b) or (d) under any governance regime. The full enumeration is in Appendix F, and Appendix G locates each convergent system.

In ML terms, what is missing is a privacy-preserving cross-tenant aggregator with an automated eval gate at the scaffold layer. Such an architecture would carry five parts: per-tenant artifact pools, each tagged with where its artifacts came from; a way to aggregate across tenants that the operator can tune for privacy; an automated eval gate that blocks a promotion whenever it does not lower patch loss; versioned lineage with RBAC; and a rollback envelope on every promoted artifact. The DP-FedAvg framing from the federated-learning literature is a useful analogy for this target, but only if artifacts are first embedded as mergeable update representations. In surveyed practice, Loop 2 is selection and review rather than parameter averaging.

Naming the missing architecture converts a vague gap into a four-property audit criterion that any future system can be measured against. Whether it *should* be filled is deployment-dependent. In safety-critical domains, reviewer-judgment gates may be a design feature rather than a defect. But no public system currently fills it, and that is what matters for the next two years of architecture work. Whether the optimizer-equipped composite reaches deployment-level reliability is a deployment-by-deployment empirical question; the paper's contribution is to name what *would have to be true* of the optimizer for that question to even be testable.

10. A new failure surface

Patchwork scaffold maintenance introduces failure modes weight-only systems do not have. Each one is also a missing piece of the optimizer.

Scaffold bloat is the dominant near-term risk. IFScale [Jaroslawicz et al., 2025] reports latency growing roughly 35× as instruction count scales from 10 to 250, and the best frontier model in that benchmark drops to 68% accuracy at 500 instructions, with weaker models degrading further. The underlying mechanism [lost-in-the-middle, instruction interference; Liu et al. [2023]] persists across model generations even when individual frontier-model scaling improves. Ratchet [Zhang et al., 2026b] supplies direct empirical evidence that bloat is also a *signal-quality* problem and not only a latency one, and that lifecycle hygiene is a precondition for residual capture and mode discovery rather than an alternative to descent (Appendix G.7). The architectural mitigation is a complexity penalty $\Omega(z)$ in the gate, explicit pruning in the loop, and hygiene operations treated as first-class.

Poisoned memory is the dominant security risk. Because memory is persistent and is read back into later prompts, an attacker who gets one malicious entry written can have it retrieved and acted on long after the attack, in sessions that never saw the original input. PoisonedRAG reports 90%+ attack success rate against standard RAG; Memory Control Flow Attacks report >90% vulnerability across major LLM agent frameworks. The mitigation is architectural too: record where each memory entry came from (provenance), gate what is allowed to be written, and keep a rollback envelope on accepted entries so a poisoned one can be removed.

Prompt injection via tools and MCP is the dominant cross-vendor attack surface. The mechanism is that a tool's output, or a connected MCP server, can smuggle instructions into the model's context that the model then follows as if the user had issued them. CVE-2025-54136 ("MCPoison") demonstrated a public MCP-server compromise pathway. The architectural mitigation is to scope each tool's permissions narrowly and to put tool bindings through the same promotion gate as any other scaffold edit, treating them as first-class scaffold coordinates rather than ambient configuration that nobody reviews.

Eval fragility is the dominant ecosystem risk. The Leaderboard Illusion documents up to 112% relative performance gains when some competitors get more access to the benchmark's own data than others — a score that reflects exposure to the test rather than true capability. Eval suites are themselves patch-local artifacts and must survive Goodhart's Law: once a metric becomes the target, optimisers learn to game it. Ong et al. [2026] extend the diagnosis to scaffold optimisers themselves: evaluating harness optimisers solely by target-agent performance gains cannot distinguish *informed* Loop-1 updates from Stage-2 trial-and-error, leaving open whether reported gains are descent (§2 Stage 3) or random search (§2 Stage 2). The architectural mitigation is held-out evals scoped within evidence radius rather than promoted as universal benchmarks, plus direct evaluation of optimiser updates against their attribution claims rather than only their downstream scores.

Coordination failures are documented in MAST: 14 failure modes, dominantly role-confusion and context-loss. The architectural mitigation is treating orchestration topology as a scaffold coordinate subject to the same gate and rollback discipline as instructions and memories.

Patch overfitting is structural, not incidental: a scaffold tuned to a deployment over-fits it, every time. The architectural mitigation is *evidence radius* discipline at promotion — do not propagate a fix beyond the scope that validated it.

Plasticity loss at the scaffold level is the slow-moving risk. Context rot, instruction conflict, memory pollution, and tool-version drift produce a scaffold-level analogue of weight-level plasticity loss. No surveyed system treats pruning or expiration as a first-class operation. The architectural mitigation is to make expiry and rollback explicit operations in the loop rather than retrospective hygiene.

The failure surface is real, quantified by 2024–2026 work, and largely unmitigated in production. Every entry on it names a piece of the optimizer that current systems do not yet have.

11. Conclusion

The field has built an external adaptation layer for LLMs, but not the optimizer that should govern it. If reliability is increasingly repaired through persistent scaffold artifacts, then production AI needs a theory and engineering discipline for the discrete, gated, auditable learning that happens there, not

just for the differentiable learning inside the model. This paper is one step toward that discipline: a survey of where production sits on the *patchwork to random-search to descent* gradient of §2, a formalisation of the Stage-3 limit under stated assumptions, and a named missing architecture where the optimizer most plainly does not yet exist.

The field did not wait for frontier weights to become perfectly reliable. It built a second learning system around them — made of markdown files, skills, memories, tools, eval suites, and rollback buttons — and what changed in 2025 and 2026 is that the combination became dense enough across vendors to read as a single external adaptation surface, most of which is still maintained as patchwork. Once that surface is named, the missing pieces become concrete engineering targets; the most consequential is governed cross-tenant scaffold optimization, and that no public system yet fills it is the survey's headline finding. The frontier model generalises. The local scaffold specialises. Reliability comes from governing that specialisation: turning today's patchwork into tomorrow's optimizer.

Appendix A. Representative survey rows

The 30 rows below are a representative spot-check of the corpus, covering all six substrates, both research and production, and all four clusters of §7. *Loop 1* and *Loop 2* columns apply to systems satisfying the composite-two-loop criterion; *n/a* means the loop is absent. *PP* is the integrated patch-local-adaptation score (0–5); *Evidence* abbreviates the strongest available source tier (T1 peer-reviewed; T2 arXiv; T3 production claim; T4 open-source artifact; T5 industry blog).

System	Year	Substrate	Loop 1	Loop 2	PP	Evidence	Why included
NanoResearch	2026	Integrated	Auto	Auto	5	T2 (arXiv:2605.10813)	Tri-level Skills/Memory/Policy co-evolution; cleanest full-auto exemplar
AutoAgent	2026	Integrated	Auto	Auto	5	T2 (arXiv:2603.09716)	Dual-cycle Execution+Evolution + elastic memory orchestration
SkillRL	2026	Integrated	Auto	Auto	5	T2 (arXiv:2602.08234)	Recursive skill-augmented RL with SR < 0.4 threshold gate
SkillOpt	2026	Integrated (S2-led)	Auto	Auto	5	T2 (arXiv:2605.23904)	Explicit deep-learning-optimizer analogy; 52/52 cells best or tied; +23.5 / +24.8 / +19.1 GPT-5.5 avg over direct/Codex/Claude Code; cross-model + cross-harness + cross-benchmark transfer
AlphaEvolve	2025	Integrated	Auto	Auto	4	T2/T3 (arXiv:2506.13131)	Production proof-point: Borg +0.7%, Gemini kernel +23%, FlashAttention +32.5%
DGM	2025	Integrated	Auto	Auto	4	T2 (arXiv:2505.22954)	Recursive self-modifying code; SWE-bench 20%→50%
ADAS	2024	Integrated	Auto	Auto	4	T2 (arXiv:2408.08435)	Meta-agent search; Turing-complete agent representation
SkillForge	2026	Integrated	Auto	Human	4	T2 (arXiv:2604.08618)	Production hybrid exemplar; LLM-judge >90% + VFS versioning
CASCADE	2025	Integrated	Auto	Human	4.5	T2 (arXiv:2512.23880)	Largest within-benchmark ablation gain: +57.9 pp on SciSkillBench
EvolveR	2025	Integrated	Auto	Auto	3	T2 (arXiv:2510.16079)	Experience-driven lifecycle; accepted at ICML 2026

System	Year	Substrate	Loop 1	Loop 2	PP	Evidence	Why included
PACE	2026	Integrated (small-model)	Auto	Auto	4	T2 (arXiv:2605.23019)	Two-timescale Loop-1 + Loop-2 self-evolution under small-frozen-model production constraints
CORAL	2026	Integrated (S5+S3 async)	Auto	Auto	4	T2 (arXiv:2604.01658)	Only async multi-agent Loop-2 mechanism in the corpus; shared persistent memory + heartbeat-gated promotion
MOSS	2026	Integrated (S2+S5 source)	Auto	n/a	4	T2 (arXiv:2605.22794)	Source-level rewriting; extends evolution to code routing / hook ordering / dispatch unreachable from the text layer
RCL	2026	Integrated (formalisation)	Auto	n/a	4	T2 (arXiv:2604.03189)	Independent formalisation of artifact-layer descent: credit assignment + $\Omega(z)$ + overfitting + plasticity in context space
GEPA	2025	(cross-substrate prompt opt)	Auto	n/a	3	T2 (arXiv:2507.19457)	Reflective prompt evolution; beats GRPO +6 avg / +20 max with 35$\times$ fewer rollouts; SkillOpt's immediate predecessor
Voyager	2023	Integrated	Auto	Auto	4	T2 (arXiv:2305.16291)	Foundational skill-library result; 3.3$\times$ items, 15.3$\times$ tech-tree milestones
AGENTS.md	2025	S1 Instructions	n/a	n/a	2	T3/T4 (agents.md)	Cross-vendor standard; 60k+ repos; Lulla 2026 measures -28.6% runtime
Claude Code CLAUDE.md	2025	S1 Instructions	n/a	n/a	4	T3/T4 (docs.anthropic.com)	Four scoping levels including org-IT policy + auto-memory layer; ceiling for S1
Cursor Rules	2024	S1 Instructions	n/a	n/a	3	T3/T4 (`docs.cursor.com/rules`)	`.cursor/rules/*.mdc` multi-scope + MCP attach; git-tracked
Anthropic Agent Skills	2025	S2 Skills	n/a	Human	3	T3 (`anthropic.com/engineering`)	Progressive disclosure spec; LangChain replication 29%$\rightarrow$95% pass-rate
SAGE	2025	S2 Skills	Auto	Auto	5	T2 (arXiv:2512.17102)	+8.9% SGC, 26% fewer steps, 59% fewer tokens on AppWorld
COSPLAY	2026	S2 Skills	Auto	Auto	5	T2 (arXiv:2604.20987)	Co-evolving decision agent + learnable skill bank; long-horizon tasks
Ratchet	2026	S2 Skills (lifecycle)	n/a	n/a	3	T2 (arXiv:2605.22148)	Hygiene-as-precondition: $+0.0$ pp without hygiene vs. $+16.2$ pp with four hygiene mechanisms; Stage-1 prerequisite for Stage-3 descent
Reflexion	2023	S3 Memory	Auto	n/a	4	T1 (NeurIPS 2023; arXiv:2303.11366)	Canonical failure-triggered episodic memory; +8% HotpotQA
Generative Agents	2023	S3 Memory	Auto	n/a	4	T1 (UIST 2023; arXiv:2304.03442)	Reflection + importance memory primitive
Cuadros et al. governed collaborative memory	2026	S3 Memory	Auto	Auto	5	T2 (arXiv:2605.04264)	Only surveyed memory system with full provenance + versioning + correction
MCP	2024	S4 Tools	n/a	n/a	5	T3 (modelcontextprotocol.io)	Cross-vendor standard; thousands of public servers; substantial enterprise adoption

System	Year	Substrate	Loop 1	Loop 2	PP	Evidence	Why included
Harvey	2026	S4 Tools	n/a	n/a	5	T3 (harvey.ai)	400K queries/day; 18,000+ workflows; 200+ legal data sources
Hippocratic AI	2026	S4 Tools	n/a	n/a	5	T3 (hippocrati-cai.com)	$3.5B valuation; 30% readmission reduction; 360% care-capacity boost
Meta-Harness	2026	S5 Orches-tration	Auto	n/a	4	T2 (arXiv:2603.28052)	End-to-end harness optimisation; filesystem-of-candidates serves as accumulated history H_t for credit assignment
Multi-Agent Evolve (MAE)	2025	S5 Orches-tration	Auto	n/a	4	T2 (arXiv:2510.23595)	RL co-evolution of Proposer/Solver/Judge population; one loop closed, no Loop-2
Cemri et al. (AG2)	2025	S5 Orches-tration	n/a	n/a	3	T2 (arXiv:2503.13657)	Specialisation +4.5 pp on GSM-Plus ($p = 0.03$); MAST 14 failure modes
Braintrust	2024	S6 Gover-nance	n/a	Auto	5	T3 (braintrust.dev)	Prompts-as-code: immutable commits, eval-gated PR merge, sub-5 min rollback
Vellum	2024	S6 Gover-nance	n/a	Auto	5	T3 (vellum.ai)	Same governance pattern; production deployment with eval thresholds
LangSmith Hub	2024	S6 Gover-nance	n/a	Auto	5	T3 (`smith.langchain.com`)	Prompt repository with environment promotion and eval blocking
Self-Refine*	2023	(contrast)	n/a	n/a	1	T2 (arXiv:2303.17651)	Fails the patch-local discriminator: in-context iteration only
Mem0*	2024	(contrast)	n/a	n/a	2	T3/T4 (mem0.ai)	Fails Loop 2: per-user memory with no cross-context promotion path
Tran & Kiela*	2026	(contrast)	n/a	n/a	1	T2 (arXiv:2604.02460)	Single-agent baselines match or beat multi-agent under matched thinking-token budgets; orthogonal finding constraining S5 claims

Asterisks mark contrast systems included for definitional clarity.

Appendix B. Data appendix

Corpus size. The survey scores approximately 130 unique LLM systems across the six substrates. Each system may be scored on multiple substrates; the per-substrate row count therefore exceeds the unique-system count.

Core / contrast split. Approximately 90 unique systems satisfy the patch-local discriminator (score ≥ 3 on at least one substrate) and form the core corpus. The remaining ~38 systems form the contrast corpus: commonly described as agentic or self-improving in public discourse but failing at least one discriminator. Appendix D enumerates representative contrast systems.

Inclusion criteria. Any system that (a) wraps a foundation model with at least one persistent artifact updated from deployment signal, or (b) is named in the public discourse as a "self-improving," "scaffolded," or "agentic" system. The contrast set (Self-Refine, ReAct, plain Mem0, DSPy/TextGrad) is included so the discriminator has something to mark against.

Exclusion criteria. Systems whose only update mechanism is weight-level fine-tuning or RLHF: these are model-side, not scaffold-side, and belong to the self-evolution literature this paper differen-

tiates from. Systems with no public evidence of deployment (white papers without code, demos, or production claim).

Evidence tiers. Each system carries one or more of: (T1) peer-reviewed publication; (T2) arXiv preprint; (T3) production claim with metrics; (T4) open-source artifact; (T5) industry-blog or interview-level documentation. T1/T4 carry highest weight in cluster analysis; T3 is admissible where the claim is concrete (specific benchmark, named integration target); T5 is admissible only when corroborated by T1–T4 elsewhere. Tier flags are propagated to cluster claims through the per-row evidence column in Appendix A; T3 production claims are not pooled with T1/T2 evidence when computing cluster-level statistics.

Evidence-tier distribution. Of the core corpus, approximately one quarter carry T1 evidence, roughly half carry T2 evidence, and the remainder carry T3 or T4 evidence as the primary tier.

Survey window and dating. January 2023 – May 2026. Preprint-only systems carry T2 evidence and are flagged in Appendix A. A fraction of T2 work is recent enough that ablations may yet be revised; cluster-level claims are robust to dropping any single T2 system.

Scoring discipline. Each system received a substrate-by-substrate 0–5 score against the rubric of §4 and an integrated patch-local-adaptation score (PP) in {0,1,2,3,3.5,4,4.5,5}. Half-scores were used sparingly when one criterion was clearly met and another partially. Ambiguous cases were resolved conservatively (downward). Scoring is single-rater; Appendix A and Appendix D together expose representative per-system scores and exclusion reasons so readers can audit ambiguous classifications independently.

Composite two-loop criterion (sensitivity analysis). The strict count of fourteen research systems satisfying the composite-two-loop criterion changes under two natural relaxations. Relaxation (a) counts any of versioning, lineage, or rollback as Loop-2 governance; the count rises to approximately eighteen by re-admitting MAE, MetaGen, MetaReflection, and CLIN. Relaxation (b) admits session signal that updates optimizer state without persisting an artifact distinct from optimiser state; the count rises to approximately sixteen by re-admitting DSPy and TextGrad. The qualitative findings (Auto$\times$Auto well-populated; [Human $\times$ Human] and [Human $\times$ Auto] empty in the surveyed corpus; governance-first cluster in [None $\times$ Auto]; the cross-tenant governed-optimisation cell empty) survive both relaxations.

Appendix C. Gate type $\rightarrow$ estimator-family mapping

The §3 formalism collapses production gates into a single per-edit gate G_t that estimates the directional loss change $\widehat{\Delta}_{z_t} L_D$. In practice, surveyed gates instantiate that abstraction in several distinct ways, and some are better read as *proposal-pruning* steps that precede the per-edit gate rather than as direct $\widehat{\Delta}$ estimators.

Gate family	What it estimates	Slot in §3 formalism	Representative systems
Eval-cascade	$\widehat{\Delta} L_D$ on a held-out task suite; cascaded through cheap-to-expensive evaluators	Per-edit gate G_t	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL-threshold	Reward improvement vs. a deployment threshold	Per-edit gate G_t	SkillRL, SAGE, EvolveR
LLM-judge with calibration	Pairwise- or rubric-scored quality, calibrated against human agreement	Per-edit gate G_t	SkillForge (Loop 1), CASCADE (consolidation gate)

Gate family	What it estimates	Slot in §3 formalism	Representative systems
Surrogate verifier	Symbolic, learned, or test-case verifier producing pass/fail or graded score	Per-edit gate G_t	DGM (sandboxed code), Voyager (in-game verifier)
Reviewer-judgment	Human credit assignment via PR review or expert sign-off	Per-edit gate G_t (human-instantiated)	SkillForge (Loop 2), Sierra Agent OS 2.0, Anthropic skills repo
Semantic-merge	Whether a candidate is semantically duplicated by, or compatible with, existing scaffold artifacts	*Proposal-pruning* preceding G_t	NanoResearch (orchestrator semantic-merge step)
MAP-Elites / archive selection	Whether a candidate populates a previously unfilled cell in a behavior-diversity archive	*Proposal-pruning* preceding G_t	AlphaEvolve (archive), ADAS (meta-agent search)
Multi-sample agreement	Whether k independent proposals or evaluations converge on the same direction; false-accept rate decays as p^k rather than variance shrinking as $1/k$	*Aggregation filter* (qualitatively distinct from mini-batch averaging)	Under-explored in the surveyed corpus

The eval-cascade, RL-threshold, LLM-judge, surrogate-verifier, and reviewer-judgment families directly populate the per-edit gate G_t of §3 and inherit its descent guarantees (under the assumptions of Appendix E). The semantic-merge and archive-selection families are not estimators of $\widehat{\Delta} L_D$. They constrain the proposal distribution rather than judging individual candidate quality, and the descent argument therefore applies to the composition (proposal-pruning $\circ$ per-edit gate). Multi-sample agreement filters are listed for completeness because they are the correct ML analogue for "wait for k pieces of evidence before accepting an edit," which is *not* the same operation as mini-batch averaging and should not be conflated with it.

Appendix D. Contrast set

The contrast corpus of approximately 38 systems is included to make the patch-local discriminator audit-able. Representative entries:

System	Year	Primary discriminator failure	Brief reason
Self-Refine [Madaan et al., 2023]	2023	No persistent artifact	In-context iteration only
ReAct [Yao et al., 2023]	2023	No persistent artifact	Thought-act-observe loop is intra-session
Reflexion [Shinn et al., 2023]	2023	No Loop 2	Per-agent episodic memory without promotion
Generative Agents	2023	No Loop 2	Per-agent reflection + importance memory

System	Year	Primary discriminator failure	Brief reason
ExpeL	2023	Loop boundary blurred	Experience bank built from training tasks
Mem0 (vanilla)	2024	No Loop 2	Per-user memory; no cross-context promotion
Zep	2024	No Loop 2	Per-agent/session knowledge graph
ChatGPT Memory	2024	No Loop 2	Per-user persistence
Claude Memory	2025	No Loop 2	Per-conversation-thread persistence
MemGPT	2023	No Loop 2	Virtual-context management per agent
DSPy	2023	No artifact distinct from optimiser state	Prompt-template optimization; SkillOpt [Yang et al. [2026]; §3.5; Appendix G] realises the artifact-bearing form.
TextGrad	2024	No artifact distinct from optimiser state	Same boundary case as DSPy; SkillOpt [Yang et al. [2026]; §3.5; Appendix G] adds the persistent `best_skill.md`, held-out validation gate, rejected-edit memory, and textual learning-rate schedule that TextGrad lacks.
MetaGen	2026	No Loop 2; no cross-session persistence	Role pool dynamic within single inference call
MAE (Multi-Agent Evolve)	2025	No Loop 2	Co-evolution within Proposer/Solver/Judge triplet
MetaReflection	2024	No Loop 2	Per-agent semantic memory
CLIN	2023	No cross-instance promotion	Causal abstractions per-agent-per-environment
LangGraph, AutoGen, CrewAI	2023–2024	Framework only	Orchestration frameworks; not deployed systems
Tran & Kiela	2026	Single-agent baseline	Constrains S5 maturity claims (see Appendix F)
W&B Weave, Agenta, LangFuse	2024	No Loop 1	Observability / eval-as-code
Mobile-Agent-E, Cradle, OS-Copilot	2024–2025	No Loop 2	Self-evolving but per-session/per-deployment
Agent S, Agent S2	2024–2025	No Loop 2	Computer-use agentic framework
ACE [Zhang et al., 2025b]	2025	No Loop 2 in surveyed form	Single-context evolving playbook

System	Year	Primary discriminator failure	Brief reason
Cursor Rules (vanilla)	2024	No Loop 2	Per-project artifacts
GitHub Copilot custom instructions, Windsurf, Continue.dev, Zed AI rules	2024	No Loop 1	Engineer-authored at repo base branch
Replit `replit.md`	2025	Loop 2 absent	Self-writing scaffold but per-workspace
Aider conventions, Lovable knowledge	2024	No Loop 1	Engineer-authored convention files

The representative entries above cover the main exclusion families (in-context-only; no Loop 2; framework-only; engineer-authored without failure trigger; optimiser-state-only); finer-grained per-system breakdowns are deferred to follow-on practitioner work.

Appendix E. Formalization of artifact-layer descent

This appendix supplies the verification machinery for §3. The body states the descent picture as a mechanism; this appendix derives the one-step descent inequality, states the convergence proposition with its assumptions, gives the full optimisation-analogy correspondence, walks four concrete cases, and clarifies what credit assignment without chain rule means.

E.1 The mechanism stack

The abstract update rule of §3 expands into six operations that every patch-local system must perform, whether explicitly engineered or accidentally emergent.

1. **Residual capture.** Log failures, corrections, rejected outputs, user edits, eval regressions, tool-call failures. The Loop-1 signal source. Without it, z_t has no provenance.
2. **Mode discovery.** Cluster repeated failures into patch-specific error modes: the *Architecture of Errors* catalogue, instantiated locally per patch.
3. **Delta synthesis.** Convert a recurring mode into a candidate scaffold edit: an instruction rewrite, a new skill, a memory correction, a tool binding, an orchestration change, an eval case. This produces z_t.
4. **Delta validation.** Estimate whether the edit reduces L_D without collateral damage on adjacent inputs. The gate G_t is Auto-gated when the check is an algorithmic criterion, Human-gated when it is reviewer judgment.
5. **Promotion control.** Decide whether the accepted edit stays in the local fork or climbs the scope hierarchy: project, stack, organisation, cross-tenant. Loop 2.
6. **Drift and bloat control.** Expire stale rules, prune conflicting memories, roll back regressed skills, detect over-fitting. Without this, accumulated edits produce the scaffold-level analogue of plasticity loss.

Each operation is the locus of a separate engineering discipline. Surveyed systems implement them partially and unevenly: research full-auto systems excel at (3) and (4); production governance leaders at (5); industry hybrids at most of (1)–(5) but rarely (6). Implementation patterns are deferred to follow-on practitioner work.

E.2 Setup and notation

Let the frontier model have fixed parameters θ_M. Let the scaffold be a structured parameter object

$$\theta_s \in \Theta_s = \Theta_1 \times \Theta_2 \times \Theta_3 \times \Theta_4 \times \Theta_5 \times \Theta_6,$$

where the six coordinates correspond to the six substrates of §4. The space Θ_s is discrete, symbolic, and versioned. For a deployment patch D, define patch loss as

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,.$$

A candidate edit $z_t \in \mathcal{A}(\theta_s^t)$ is drawn from a proposal process $z_t \sim Q_t(z \mid H_t, \theta_s^t)$, where H_t is the accumulated history of failures, corrections, rejected outputs, eval regressions, reviewer comments, tool-call traces, and prior accepted deltas, and conditioning on θ_s^t ensures admissibility at the current scaffold. Q_t is not uniform over admissible edits: it concentrates on the failure modes with the largest expected loss contribution. For a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, that contribution is $f_m s_m$ — repeatability times severity — so high-frequency or high-severity modes are proposed first. This is the proposal-side counterpart to the gate's acceptance test, and what makes A3 (proposal coverage), stated below, plausible rather than merely assumed. The finite directional difference induced by the edit is

$$\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s).$$

A useful edit is one with $\Delta_z L_D < 0$. The gate's noisy estimate of this quantity is $\widehat{\Delta}_z L_D$.

E.3 Descent inequality

The gate operates on the *estimator*, not the truth. The descent argument distinguishes them.

Assumption A1 (positive descent bias on accepted edits). There exists $\gamma_t > 0$ such that

$$\mathbb{E}\Big[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t\Big] \le -\gamma_t - \tau_t.$$

The gate's accept-threshold τ_t ensures accepted edits clear the margin under the estimator.

Assumption A2 (bounded estimator bias on accepted edits). There exists $\varepsilon_t \ge 0$ such that

$$\big|\mathbb{E}\Big[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t\Big]\big| \le \varepsilon_t.$$

Combining A1 and A2 with the gated update rule of §3 and writing $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}\big[L_D(\theta_s^{t+1}) \mid \theta_s^t\big] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\varepsilon_t.$$

This is descent under a noisy estimator: per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability.

E.4 Convergence proposition

Proposition (convergence of loss, informal). Assume A1, A2, bounded loss $0 \le L_D \le B$, and additionally:

- **A3 (proposal coverage).** At every non-G-stable scaffold there exists $\delta > 0$ such that Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \le -\delta$.
- **A4 (effective compactness).** The reachable scaffold orbit takes values in a finite or precompact subset of Θ_s.
- **A5 (persistent acceptance with summable bias).** $\sum_t p_t(\gamma_t + \tau_t) = \infty$ and $\sum_t p_t\varepsilon_t < \infty$.

Then $L_D(\theta_s^t)$ is a non-negative supermartingale up to summable noise; $L_D(\theta_s^t) \to L_\infty$ almost surely for some random $L_\infty \ge 0$; and the limit set of $\{\theta_s^t\}$ is G**-stable**: no admissible edit clears the gate with negative expected directional loss in the limit.

The proposition claims convergence of the *loss* and stability of the *gate decision*, not convergence to a global or local optimum of L_D. A globally suboptimal G-stable scaffold is consistent with the result: Q_t may never propose the improving edit, or $\widehat{\Delta}$ may be biased to reject an improving direction. This is the ordinary target of local stochastic optimization under noisy estimators rather than a guarantee of optimality.

E.5 Optimisation-analogy correspondence

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, eval gates, governance rules
Forward pass	Model–scaffold composite executing on patch input
Loss	Failure, user correction, eval regression, retrieval miss, tool error, schema violation
Finite directional difference	$\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$
Gradient estimate	Credit assignment identifying the scaffold coordinate responsible for the failure
SGD step	Accepted scaffold edit
Aggregated update	Multiple recurring failures aggregated before update (either by sampling, *mini-batch averaging*, or by sign-of-direction agreement across k proposals, *multi-sample voting*)
Learning rate	Edit magnitude under $\oplus$
Update radius	Promotion scope: which loss is being descended on (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch eval before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this patch and degrades elsewhere
Rollback	Reverting a harmful accepted update

E.6 Credit assignment without chain rule

Scaffold systems do not compute chain-rule derivatives through markdown files or tool registries, and the correspondence above deliberately omits a "backpropagation" row. They do, however, perform *credit assignment* through a pipeline. If the final answer fails, the responsible coordinate may be a retrieval configuration three layers upstream, a missing tool schema constraint, an underspecified instruction, a stale memory, or a bad orchestration edge. Engineers do this manually; LLM-judges and surrogate verifiers do it semi-automatically. The operation is *attribution* (output error is assigned to an upstream coordinate) rather than backpropagation. The descent argument of §3 needs only that this attribution succeeds with non-zero probability per failure (assumption (iv) of the scope box), not that it is performed by chain rule.

E.7 Worked cases

RAG. Suppose failures recur because retrieved context omits the decisive clause. The observed loss is not "the model hallucinated." Credit assignment traces the error to the retrieval coordinate: chunking, query rewrite, metadata filter, embedding model, reranker, citation policy, or context-packing rule. The candidate edit might tighten chunk boundaries, add a metadata constraint, change the reranker, or insert a regression eval containing the missed clause. The gate tests whether retrieval recall and answer accuracy improve on the patch eval without degrading adjacent cases. If accepted, the scaffold has moved downhill on the local loss surface.

Pitch review. Suppose the system repeatedly overrates decks claiming a "huge market" without numeric evidence. The failure cluster identifies an under-specified rubric coordinate. The candidate edit changes "evaluate TAM size" into "require a numeric market estimate and source; if absent, cap TAM score at 4/10." The gate runs the revised rubric against held-out decks with human scores. If agreement improves without damaging unrelated criteria, the edit is committed.

Tool-using agent. Suppose calls repeatedly fail because optional parameters are underspecified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The candidate

edit adds required fields, constrained decoding, or a pre-call uncertainty check. The gate estimates whether tool-call failure decreases without increasing refusal, latency, or unnecessary clarification.

Memory. Suppose a stale customer preference is repeatedly retrieved and causes bad decisions. The failure is not a weight error. The responsible coordinate is a memory entry plus its provenance, priority, and expiry policy. The candidate edit corrects, expires, or version-tags the memory. The gate checks whether the correction improves future behavior without erasing useful history.

E.8 Loop 1 and Loop 2 as descent steps of different radius

Loop 1 proposes and validates local descent steps on a single L_D. Loop 2 expands the update radius. It selects, merges, and promotes accepted local edits so that they apply to a larger averaging set of patch losses. USER-level acceptance descends on a single L_D. PROJECT promotion descends on a project-averaged loss. STACK promotion widens the average further. CORE or cross-tenant promotion descends on a federated average. The two-loop design space is therefore not just a taxonomy of engineering patterns: it is a taxonomy of *which loss each surveyed system is descending on*, gated by which estimator.

E.9 Overfitting in scaffold adaptation

In weight training, overfitting is usually a defect. In scaffold adaptation, local overfitting is partly the point. Frontier weights are trained to preserve cross-patch generality; they must smooth over local conventions and rare workflow-specific failures. A deployment patch needs the opposite: selective specialization to local residuals. The scaffold is therefore an intentionally overfittable layer, but one whose overfitting is scoped, inspectable, versioned, gated, and reversible. A scaffold that improves one repository, hospital workflow, or legal diligence process may degrade performance elsewhere. That is not a contradiction; it is patch adaptation. The mistake is promoting that edit beyond its evidence radius.

In non-stationary patches ($D = D_t$), the same descent mechanism tracks a moving optimum rather than converging once. This is why recency weighting, replay, pruning, expiry, and rollback are not optional hygiene; they are the scaffold-level analogues of continual-learning machinery.

Appendix F. Survey detail

This appendix carries the cell-by-cell discussion of the two-loop matrix from §8, the closest-approaches enumeration that supports §9, and the deployment-dependence argument. It supplies the verification material for §§8–9.

F.1 Two-loop matrix: cell-by-cell discussion

[Auto $\times$ Auto] is most populated (twelve systems). Every member has a machine-evaluated decision criterion on both loops. Velocity is the cluster's defining property; the lack of cross-organisational governance is its uniform weakness.

[Auto $\times$ Human] is the production-viable cell: Loop 1 fires at machine speed under an algorithmic criterion, Loop 2 requires human approval before propagation. The pattern reconciles automation velocity with reviewer accountability. SkillForge's LLM-judge + reviewer pipeline is the architectural exemplar. CASCADE, Sierra, and Devin instantiate variants; the industry exemplars (Sierra, Devin) are marked with † because vendor documentation rather than peer-reviewed metrics is the primary evidence.

[None $\times$ Auto-gated] holds the governance-first prompts-as-code systems (Braintrust, LangSmith Hub, Vellum). They have a real Loop 2 (eval-gated CI blocks promotion on regression), but no Loop 1 in the patch-local sense, because the candidate artifacts are authored by engineers at their desks rather than triggered by session feedback. The asterisks mark this: governance without inner-loop adaptation. The cell is informative because it shows what survives when only the outer loop is automated.

[Human $\times$ Auto] is empty by engineering logic: if humans author candidates manually, automated promotion offers little leverage. A speculative occupant would be "expert curation + auto-distribution"

(expert-authored skills automatically gated through a held-out eval before shipping); no published example exists.

[Human × Human] is empty in the surveyed corpus. A candidate occupant would be a reviewer-authored-and-reviewer-promoted artifact pipeline targeting solo and small-team workflows where automation cost exceeds review latency. No public example was found during the May 2026 survey window.

[Human × None] is structurally improbable: a system requiring human authorship on Loop 1 but with no Loop 2 collapses to "engineer-edits-a-config" and is not patch-local in the sense of §3.

DGM dual-mode note. DGM operates in both [Auto × Auto] (primary archive-based evolution) and [Auto × Human] (sandboxed code-modification review) modes depending on deployment configuration; we list it once in the dominant cell.

F.2 Cross-cutting findings

Noise reduction via batching is largely absent in the surveyed auto-gated systems. Every auto-gated system fires at $n = 1$ per generation step, absorbing estimator noise statistically via the per-edit gate rather than aggregating evidence across multiple proposals. SkillOpt is a partial exception: it explicitly batches evidence (rollout batch B and reflection minibatch B_m=8) and decays edits via a learning-rate schedule, so the noisy-estimator absorption is done by aggregation as well as by the per-edit gate; the n=1 characterisation still holds for the *single accepted edit per step*, but the proposal evidence is aggregated. Multi-sample agreement filters (accept only when k independent proposals converge) are a tractable design dimension that no surveyed system has formalised. Their noise-reduction behaviour is qualitatively distinct from mini-batch averaging (false-accept rate shrinks roughly as p^k rather than variance shrinking as $1/k$), and the two should not be conflated.

Multi-layer hierarchies are under-explored. Only AlphaEvolve demonstrates explicit depth in the auto-gated cluster (via cascade evaluator and MAP-Elites archive); the other twelve composite-two-loop systems treat their artifact pool as flat.

F.3 Closest approaches to governed cross-tenant scaffold optimization

The four properties of governed cross-tenant scaffold optimization are (a) auto-gated Loop 1, (b) multi-tenant cross-org promotion, (c) versioned lineage with RBAC, (d) rollback. Closest approaches partition the requirements:

- **AlphaEvolve** has (a) and partial (b) within a single organisation; no cross-org promotion mechanism.
- **NanoResearch** has (a) at the research level; no enterprise governance layer described.
- **SkillOpt** has (a) fully and partial (c) via the exported `best_skill.md` artifact plus protected slow-update field; reports positive cross-model, cross-harness, and cross-benchmark transfer as empirical evidence-radius measurement; does not implement (b) cross-organisational promotion or (d) explicit rollback. The Appendix G walkthrough maps every SkillOpt design decision to its §3 counterpart.
- **CORAL** [Qu et al., 2026] has partial (a) via heartbeat-gated promotion and is the only system in the corpus that implements asynchronous multi-agent Loop 2 within one organisation through shared persistent memory; no (b) cross-organisational scope, no formal (c) RBAC, no (d) rollback.
- **PACE** [Ling et al., 2026] has (a) at the small-model production timescale and a fast/slow two-timescale Loop 1 + Loop 2 that consolidates across one deployment's own episodes rather than across deployments; no (b), no formal (c) or (d). PACE is the closest demonstration that the two-loop architecture is implementable under small-frozen-model constraints.
- **Meta-Harness** [Lee et al., 2026] has (a) over the harness substrate (S5) with a filesystem-of-candidates credit-assignment record that functions as the accumulated history H_t of §3; no Loop-2 cross-organisational promotion, no RBAC, no rollback.
- **SkillForge** has (a) and (b) within one enterprise, partial (c) via VFS commits; (d) is undocumented in the public report.
- **Sierra Agent OS 2.0** has (b) and partial (a); the GitHub-style Workspaces provide some lineage but not RBAC-with-rollback in the strict sense.

- **Vellum / Braintrust / LangSmith Hub** have (c) and (d) explicitly but no (a) or (b) in the strict sense; these are governance-first systems by construction.

No surveyed system combines (b), (c), and (d) under any gate type. The cross-organisational governance triad is itself unfilled, independently of whether Loop 1 is auto-gated.

F.4 Deployment-dependence of the missing architecture

Whether the corner *should* be filled is deployment-dependent. In safety-critical domains (medical decision support, legal advice, financial advice), reviewer-judgment gates may be a design feature rather than a defect, and the human latency is the point. The narrower survey claim is what matters: no public system currently combines automated local scaffold evolution with governed cross-organisation promotion, versioned lineage with RBAC, and rollback. Naming the corner converts a vague gap into a specific four-property audit criterion that any future system can be measured against. The shortest demonstrable path appears to build on AlphaEvolve's cascade evaluator and MAP-Elites archive, extending federation from within-DeepMind multi-target to cross-customer multi-tenant. The architectural surface, not the timeline, is what this paper claims.

Appendix G. SkillOpt walkthrough and the convergent cluster

This appendix expands §3.5 along two axes. The first six subsections (G.1–G.3 on SkillOpt; G.4–G.8 on the convergent cluster) walk each system mechanism-by-mechanism, stating its §3 mapping explicitly and naming what it leaves open. The next subsection (G.9) synthesises four claims supported by the cluster as a whole. The final two subsections (G.10–G.11) carry the §9-property audit of SkillOpt and the published cost-per-point figure.

G.1 The convergent landscape

SkillOpt [Yang et al., 2026] is the cleanest current instance of artifact-layer descent in our survey, but it is one of at least fourteen independent research threads in 2025–2026 that converge on the same disciplined gradient-like update mechanism for the scaffold layer. The cluster is striking because the convergence happens *from different starting points*: GEPA and Tiwari et al. start from prompt-optimisation and continual-learning respectively; RCL starts from cognitive-agent learning; Meta-Harness, AHE, MOSS, CANTANTE, and Ong et al. start from harness engineering; MOCHA and Ratchet start from skill-deployment constraints; ICT, Combee, CORAL, and PACE start from cross-task / multi-agent / production-deployment angles. Each system instantiates a different subset of §3's five conditions, and each exposes a different gap.

The table below indexes the cluster by §3 mechanism and by what each system leaves open. Subsequent subsections walk each cluster in detail.

Cluster	Representative systems	What it instantiates in §3	Substrate (§4)	What it leaves open
Reflective-evolution	GEPA [Agrawal et al., 2025]; Tiwari et al. [2026]	Q_t over trajectory evidence with Pareto-frontier proposal-pruning; two-timescale in-context / in-weights motivation	(cross-substrate prompt opt)	persistent θ_s; L_t budget; held-out G_t; rejected-edit buffer

Cluster	Representative systems	What it instantiates in §3	Substrate (§4)	What it leaves open
Reflective context	RCL [Vassilyev et al., 2026]	θ_s over context space; generalisation penalty = $\Omega(z)$; explicit naming of credit-assignment / overfitting / plasticity-loss in context space	S1+S2 (context)	cross-deployment Loop 2
Harness optimisation	Meta-Harness [Lee et al., 2026]; AHE [Lin et al., 2026]; MOSS [Cai et al., 2026]; CANTANTE [Zehle, 2026]; Ong et al. [2026]	credit assignment (§3 op 3); filesystem-of-candidates as H_t; contrastive multi-agent attribution; source-level extension beyond text-mutable	S5 (orchestration); S2+S5 (MOSS)	governed Loop 2 across orgs; RBAC; rollback
Multi-objective + lifecycle	MOCHA [Tanjim et al., 2026]; Ratchet [Zhang et al., 2026b]	Pareto frontier as the true shape of $\Omega(z)$; hygiene as precondition for residual capture and mode discovery	S2 + deployment constraints	gradient-analogue closure; gate strictness
Cross-task transfer	ICT [Shalev et al., 2026]; Combee [Li et al., 2026]; CORAL [Qu et al., 2026]; PACE [Ling et al., 2026]	empirical evidence-radius (Loop-2 scope) measurement; async multi-agent Loop 2 within one org; two-timescale Loop 1 + Loop 2 under small-model constraints	Integrated; S3+S5	cross-organisational gate, lineage, rollback
Worked instance	SkillOpt [Yang et al., 2026]	all five §3 conditions: θ_s, L_D, Q_t/z_t, L_t, G_t + rejected-edit buffer + slow/meta update	S2-led Integrated	cross-org Loop 2; RBAC; rollback

Read against §9, no member of the cluster combines (a) auto-gated Loop 1 with (b) cross-organisational Loop 2, (c) versioned lineage with RBAC, and (d) rollback. The §3 picture is realised in fragments across the cluster; the §9 gap is confirmed by every member.

G.2 SkillOpt: full correspondence

The table below extends the §3.5 compact correspondence with the §3 role of each component and an algorithm-level pointer into SkillOpt's published procedure (their Algorithm 1).

§3 quantity	SkillOpt instantiation	§3 role explained	Algorithm pointer
θ_s (scaffold)	`best_skill.md` — a 379–1,995-token markdown document	the trainable external state of the frozen target model; persistent and exported as the deployment artifact	s_{best} (Algorithm 1, lines 1, 22, 37)
L_D (patch loss)	benchmark-native hard score / exact-match on a held-out test split	scalar quality signal that the optimiser is implicitly minimising	EVALUATE(M, h, s, D) (lines 2, 17, 37)
Q_t (proposal)	optimizer-model reflection over failure and success minibatches; failures and successes analysed separately, then merged with priority on failure fixes	proposal distribution conditioned on accumulated rollout evidence and current scaffold	optimizer calls O for failure analysis, success analysis, and merge (lines 9–11)
z_t (edit)	structured add/delete/replace patch on the skill (or rewrite-from-suggestions in rewrite mode)	the candidate update applied via $\oplus$ to the current scaffold	rank/clip to L_t edits, apply to obtain $\tilde{s}$ (lines 12–13)
L_t (textual learning rate)	bounded per-step edit budget; constant / linear / cosine / autonomous schedules; default cosine with floor $L_t = 2$	edit-magnitude controller; the §3 *learning rate* in textual form	rank/clip step (line 12); schedule applied per epoch
G_t (gate)	held-out selection-split accept gate, strictly greater than current selection score	the per-edit gate G_t — accepts only when $\widehat{\Delta}_{z_t} L_D$ improves the selection score	comparison at line 20; cache lookup at lines 14–18
rejected-edit buffer	epoch-local store of failed proposals and the score drop they caused	converts rejected steps into negative feedback for later proposals within the same epoch; zero deployment cost	$\mathcal{B}$ updated at line 26; reset at line 5
slow/meta update	epoch-end longitudinal guidance written to a markup-fenced protected slow-update field, still passed through D_{sel}	second-order consolidation across adjacent epochs; the §3 "step-level edits cannot overwrite the protected slow-update field" decomposition	lines 28–31; gate applied to injected guidance
optimizer-side meta skill	m_{meta} — a prompt-side summary of which edits helped, which failed, and which failures persisted	training-time guidance for Q_t; never shipped with the deployed artifact	lines 33–34 (training-side only)

The mapping is exact rather than analogical: every §3 quantity has a named, code-level counterpart in SkillOpt's Algorithm 1, and every SkillOpt control surface maps onto a §3 quantity. SkillOpt is therefore the cleanest published instance we are aware of of the §3 picture as *mechanism* rather than

metaphor — the five conditions of §3 (measurable patch loss, residual capture, credit assignment, candidate delta synthesis, directional-loss-change gate) are all instantiated.

G.3 What SkillOpt does that prior textual-optimisation systems do not

SkillOpt's own experiment design compares it against six baselines under an identical protocol (same target model, same held-out test split, same scorer). The differentiation below frames each baseline in §3 vocabulary and identifies the §3 mechanism the baseline lacks. The through-line is the paper's thesis: *the scaffold/evolution layer needs a disciplined gradient-like update mechanism*, and the live distinction across baselines is which §3 conditions each one fails.

- **TextGrad** [Yuksekgonul et al., 2024]**.** Backpropagates textual feedback through `tg.Variable` objects in a compound system. In §3 terms: z_t-style edits exist, but θ_s does not — TextGrad has no artifact distinct from optimiser state. There is no L_t schedule, no held-out G_t, and no rejected-edit buffer. SkillOpt's measured advantage over TextGrad on direct chat ranges from $+5.9$ to $+34.0$ points across benchmarks at GPT-5.5 scale and is uniformly positive at all seven target-model scales. This matches our Appendix D classification of TextGrad as failing the patch-local discriminator on persistence; SkillOpt closes precisely that gap.
- **GEPA** [Agrawal et al., 2025]**.** Reflective prompt evolution with Pareto-style multi-candidate selection. Closer than TextGrad in that it carries multiple candidates and a selection step. In §3 terms: Q_t is real, but the optimisation target is a prompt rather than a persistent skill artifact, G_t is not the strictly-greater held-out gate of §3, and there is no L_t bound — accepted edits can wholesale rewrite. SkillOpt outperforms GEPA in the reported comparison; the per-cell deltas should be read directly from SkillOpt's results table rather than summarised here.
- **Trace2Skill** [Ni et al., 2026]**.** Distils trajectory-local lessons into reusable skill artifacts. In §3 terms: θ_s exists, but there is no iterative G_t — Trace2Skill mines skills offline rather than running a held-out validation loop that accepts or rejects each candidate. Our §3 mechanism stack therefore loses *delta validation* (operation 4) and the *rejected-edit buffer*. SkillOpt's measured gap over Trace2Skill is $+5.0$ to $+19.0$ points across benchmarks at GPT-5.5 direct chat.
- **EvoSkill** [Alzubi et al., 2026]**.** Skill-folder evolution under failure analysis, deployed harness-side. The strongest baseline on the Codex and Claude Code harness rows. In §3 terms: θ_s is a folder rather than a single artifact, and credit assignment via failure analysis is genuine, but there is no L_t bound on per-step folder edits and no rejected-edit buffer. SkillOpt beats EvoSkill by $+14.0$ inside Codex and $+3.2$ inside Claude Code on the GPT-5.5 five-benchmark average.
- **Human skill** (expert-written, 145–516 tokens per benchmark). In §3 terms: θ_s exists, but Loop 1 does not — the artifact is static at deployment. Provides the no-Loop-1 ceiling.
- **One-shot LLM skill** (generated zero-shot from a benchmark description by GPT-5.5, never updated). In §3 terms: θ_s exists; Loop 1 does not; Q_t fires exactly once with no rollout evidence. The "untrained scaffold" baseline.

ABSTRAL [Song et al., 2026] and EvoTest [He et al., 2025], referenced in SkillOpt's related work but not measured in their table, operate at the multi-agent design / test-time configuration layer rather than the persistent skill layer; they are out of scope for the artifact-layer-descent comparison.

Synthesising across all six measured baselines: the live difference is whether the loop has (i) a persistent edit-target distinct from optimiser state (θ_s), (ii) a held-out validation gate on each accept (G_t with $\widehat{\Delta}$ estimator and accept threshold τ_t), (iii) negative-feedback retention (rejected-edit buffer feeding Q_t), and (iv) a bounded learning rate with a schedule (L_t + cosine/linear/constant/autonomous). SkillOpt is the first published system in our survey that instantiates all four simultaneously, and it does so as a single offline training loop whose deployed artifact remains a 300–2,000-token markdown file requiring no inference-time optimiser calls.

G.4 GEPA, Tiwari, and the reflective-evolution family

The immediate predecessor to SkillOpt's optimiser framing is **GEPA** [Agrawal et al., 2025]. It shows that natural-language reflection over execution trajectories can provide a richer adaptation signal than sparse scalar rewards for prompt evolution in compound AI systems. GEPA samples trajectories containing reasoning steps, tool calls, and tool outputs; reflects on them to diagnose failures and propose prompt updates; and maintains a Pareto frontier of candidate prompts rather than greedily mutating only the current best. Across six tasks, GEPA outperforms GRPO by 6% on average and by up to 20% while using up to 35$\times$ fewer rollouts, and also outperforms MIPROv2 in the reported aggregate comparison. In §3 vocabulary, GEPA strongly instantiates the proposal side, Q_t, through trajectory-conditioned reflection, and uses Pareto-based candidate selection as a proposal/pruning mechanism. What it does not instantiate is the full SkillOpt-style artifact lifecycle: a bounded edit budget on a reusable skill document, a strictly-greater held-out gate on each accepted skill update, a rejected-edit buffer used as negative feedback, and an exported deployment artifact with governance hooks.

Tiwari et al. [2026] supply the broader theoretical framing from the optimisation side, arguing explicitly that restricting learning to *either* in-context (prompt optimisation) *or* in-weights (RL / fine-tuning) is unnecessary and costly. In-context adaptation is cheap and rapid but capacity-bounded; in-weights adaptation is expressive but induces catastrophic forgetting and loss of plasticity [Dohare et al., 2024]. The scaffold layer of §2 is precisely the substrate that absorbs what in-context adaptation can carry cheaply, leaving the weights free to generalise. Tiwari et al. supply the two-timescale motivation from the optimisation side; §2 supplies the matching motivation from the production-reliability side. The two framings are independent and mutually reinforcing.

G.5 Reflective Context Learning: a parallel formalisation

The most direct independent convergence on the §3 formalisation comes from **Reflective Context Learning** [RCL; Vassilyev et al. [2026]]. Their opening observation is that "the fundamental problems of learning, including credit assignment, overfitting, forgetting, local optima, and high-variance learning signals, persist whether the learned object lies in parameter space or context space," and that current methods are "fragmented and ad hoc" precisely because they address these challenges one at a time rather than as a unified optimisation problem over the context. The parallel to §3's argument — that scaffold maintenance needs the discipline of weight-space optimisation applied to an external artifact layer — is exact and arrived at independently.

RCL maps onto §3 as follows. The *learned context* is θ_s. The *reflection signal* is a form of credit assignment identifying which context coordinate caused a failure (§3 operation 3). The *generalisation penalty* that RCL introduces to prevent narrow sample-specific rules is structurally the complexity penalty $\Omega(z)$ of the gate condition. Where RCL and §3 differ is in Loop-2 scope: RCL is framed around single-agent cross-task generalisation, while §3's Loop 2 is cross-deployment, cross-tenant, and cross-organisational promotion. The convergence is nonetheless significant: two research threads starting from different motivations (production reliability in §3, cognitive agent learning in RCL) arrive at the same formal observation that the classical learning problems do not disappear when the learned object moves outside the weights.

G.6 The harness-optimisation cluster

A second convergent cluster focuses on the *harness* — the code and configuration layer that wraps a frozen model — rather than on skill documents specifically. This cluster independently identifies credit assignment as the central unsolved problem of scaffold evolution, and its engineering solutions directly instantiate §3's operation 3 (mode discovery) and operation 4 (delta validation).

Meta-Harness [Lee et al., 2026] is the cleanest representative. Its opening claim — that "the performance of large language model systems depends not only on model weights, but also on their harness: the code that determines what information to store, retrieve, and present to the model" — is §1's scaffold-versus-weights decomposition restated for the harness substrate specifically. Meta-Harness then identifies that existing text optimisers "compress feedback too aggressively," losing the fine-grained signal needed for credit assignment. Its solution is an outer-loop proposer with access to source code, scores, and execution traces of all prior candidates through a filesystem, so that the edit history itself becomes the credit-assignment signal. In §3 terms, the filesystem of prior candidates is

a concrete form of the accumulated history H_t that conditions the proposal distribution Q_t, and the outer loop is a Loop-1 closure over the harness substrate (§4, S5) rather than the skill substrate (S2).

Agentic Harness Engineering [AHE; Lin et al. [2026]] attacks the same problem from the observability side. The core diagnosis is that harness engineering "faces a heterogeneous action space across editable components, voluminous trajectories that bury actionable signal, and edits whose effect is hard to attribute." These three failure modes map precisely onto §3's mechanism stack: a heterogeneous action space is the coordinate-identification problem (op 2, mode discovery); voluminous trajectories that bury signal is the residual-capture problem (op 1); and effects that are hard to attribute is operation 3, credit assignment. AHE's response — three matched observability pillars giving every editable harness component a file-level representation — is an engineering instantiation of the coordinate decomposition that §3 assumes. The implicit claim is that artifact-layer descent requires not just the optimisation loop but also the instrumentation that makes scaffold coordinates individually observable and attributable.

MOSS [Cai et al., 2026] extends the harness argument by observing that all prior self-evolving agent systems "confine evolution to text-mutable artifacts — skill files, prompt configurations, memory schemas, workflow graphs — and leave the agent harness untouched." Since routing, hook ordering, state invariants, and dispatch live in code rather than in any text artifact, an entire class of structural failure is, in MOSS's phrase, "physically unreachable from the text layer." MOSS therefore extends Loop-1 evolution to source-level code rewriting. In §4 terms, MOSS evolves S2 (Skills) and S5 (Orchestration) simultaneously, rather than treating the orchestration substrate as a fixed wrapper around an evolvable skill layer. This matters for §9: a governed cross-tenant scaffold optimiser that covers only text-mutable artifacts leaves the code substrate ungoverned, and MOSS's diagnosis suggests the gap is not merely inconvenient but structurally incomplete.

CANTANTE [Zehle, 2026] attacks credit assignment directly in the multi-agent setting. Its observation is that "scores are available only at the system level, whereas the parameters governing agent behavior are local," making optimisation of multi-agent scaffolds "fundamentally a credit-assignment problem." CANTANTE's solution is contrastive: it decomposes system-level rewards into per-agent update signals by contrasting rollouts of multiple jointly acting configurations. In §3 vocabulary, CANTANTE provides a concrete mechanism for operation 3 in the case where the scaffold is distributed across agents rather than concentrated in a single artifact. The gap it closes is the same one §3 identifies in Appendix E.6 — that attribution (output error assigned to an upstream scaffold coordinate) must succeed with non-zero probability per failure — but instantiated for the multi-agent topology substrate (S5).

Harness optimizer evaluation [Ong et al., 2026] provides independent empirical evidence for a failure mode this paper flags in §10: that evaluating scaffold optimisers "solely by observing target agents' performance gains" misses intermediate erroneous edits that hinder performance. Their finding — that it is "unclear whether harness optimization is driven by optimizers' informed update actions or simply trial-and-error" — is a direct empirical instantiation of the distinction between Stage 2 (random artifact search) and Stage 3 (artifact-layer descent) from §2's table. The credit-assignment problem is not merely theoretical: production harness optimisers that cannot distinguish informed updates from lucky trial-and-error are operating in Stage 2 even when they report benchmark improvements.

G.7 Multi-objective and lifecycle constraints

Two papers expose constraints on the §3 picture that single-loss descent does not capture, both relevant to the §9 missing-architecture discussion.

MOCHA [Tanjim et al., 2026] observes that agent skills are "multi-field artifacts subject to hard platform constraints: description fields are truncated for routing, instruction bodies are compacted via progressive disclosure, and co-resident skills compete for limited context windows." Skill optimisation is therefore not a single-loss descent problem but an inherently multi-objective one: a skill must simultaneously maximise task performance and satisfy platform limits, and optimising for task performance alone produces skills that pass benchmarks but violate deployment constraints. MOCHA introduces *Chebyshev annealing* to navigate the constraint frontier. In §3 terms, the complexity penalty $\Omega(z)$ in the gate condition is precisely the mechanism MOCHA is operationalising, but §3's scalar Ω is a simplification: in production, the constraint surface is a Pareto frontier, not a single penalty term. MOCHA's empirical contribution is to show that ignoring this frontier — treating skill

optimisation as unconstrained single-loss descent — produces skills that degrade on real deployment infrastructure even when benchmark scores improve.

Ratchet [Zhang et al., 2026b] makes an argument that is, in some respects, orthogonal to the gradient-analogue framing and for that reason worth engaging directly. Ratchet's opening finding — that LLM-authored skills deliver $+0.0$ pp over no-skill baselines while human-curated ones deliver $+16.2$ pp — suggests that the bottleneck in self-evolving skill libraries is not the optimisation algorithm (the mechanism for proposing and accepting edits) but *lifecycle management*: whether skills are written, retrieved, curated, and retired correctly. Ratchet introduces four hygiene mechanisms — outcome-driven retirement, a bounded active-cap, meta-skill authoring, and pattern canonicalisation — and shows that these lift the $+0.0$ pp floor substantially before any gradient-analogue optimiser is applied.

This finding is not a refutation of the §3 picture but a clarification of the precondition for it. Artifact-layer descent, as formalised in §3, presupposes that the scaffold is maintained well enough for the gate G_t to have a meaningful signal: if skill retrieval is so noisy that the wrong skill is injected into every rollout, the proposal process Q_t receives corrupted evidence and the descent guarantee degrades to random search. Ratchet's hygiene mechanisms are, in §3 vocabulary, prerequisites for operation 1 (residual capture) and operation 2 (mode discovery) to function: outcome-driven retirement prevents stale artifacts from polluting the residual signal; the bounded active-cap prevents scaffold bloat from making retrieval unreliable (the §10 failure mode); pattern canonicalisation collapses duplicate and near-duplicate skills so the proposal process conditions on a clean, deduplicated artifact set rather than competing variants of the same pattern. The practical implication is that the deployment path to Stage-3 artifact-layer descent runs through Stage-1 hygiene as a *prerequisite*, not as an alternative. A system with strong hygiene and no optimiser sits at Stage 1 with a clean scaffold; a system with a strong optimiser and poor hygiene sits at Stage 2 with a noisy loss signal. Stage 3 requires both.

G.8 Cross-task transfer and evidence radius

Several papers independently measure what §3 calls the *evidence radius* of a scaffold artifact — how far an edit validated in one context continues to improve performance when redeployed elsewhere — without using that vocabulary.

Training Language Agents to Learn from Experience [ICT; Shalev et al. [2026]] introduces the In-context Training framework specifically to evaluate *cross-task self-improvement*: whether experience distilled from one set of tasks transfers to unseen tasks. Their finding that "current reflection-based methods can only self-correct within a single task instance" is a measurement of evidence radius at its lower bound — a radius of exactly one task — and their ICT framework is a systematic attempt to expand that radius through supervised distillation of reflections into reusable lessons. In §3 terms, ICT is an attempt to instrument Loop 2 empirically: to measure whether a locally-validated edit (one that passed the gate on one task's L_D) continues to improve performance when the promotion radius is expanded to cover a distribution of unseen tasks.

Combee [Li et al., 2026] approaches the same measurement from the scaling direction. ACE [Zhang et al., 2025b] established that context-as-living-artifact (a system prompt that accumulates, refines, and organises strategies through agentic interaction) could improve over static prompts on single-agent tasks. Combee extends this to high-parallelism settings, arguing that "existing methods primarily focus on single-agent or low-parallelism settings," which "fundamentally limits their ability to efficiently learn from a large set of collected agentic traces." The parallel scaling Combee addresses is a form of Loop-2 scope expansion: when many agents run simultaneously, the trace pool available for the next proposal step is drawn from a wider deployment distribution, and the promoted artifact is being tested against a larger averaging set of patch losses. This is not cross-organisational Loop-2 promotion, but it is a step toward it, and Combee's empirical results measure how much of the evidence-radius benefit can be captured by intra-organisation parallelism before the cross-organisational barrier is reached.

CORAL [Qu et al., 2026] addresses the Loop-2 problem from the multi-agent open-ended discovery angle. CORAL replaces rigid control with long-running agents that explore, reflect, and collaborate through shared persistent memory, asynchronous multi-agent execution, and heartbeat-based interventions. The shared persistent memory is the cross-agent promoted artifact: a finding validated in one agent's exploration that is written to the shared store becomes available to all agents, and the heartbeat mechanism provides a form of auto-gated Loop-2 promotion. CORAL does not implement

the versioned lineage, RBAC, or rollback that §9 requires, but it provides the only fully asynchronous multi-agent Loop-2 promotion mechanism in the current survey corpus, and its open-ended discovery framing makes the evidence-radius question concrete.

PACE [Ling et al., 2026] closes this cluster with a production-oriented two-timescale design for small language model agents. The fast timescale adapts prompts, parsers, validators, and control logic from failure signals within a deployment; the slow timescale consolidates across episodes. The two-timescale structure maps cleanly onto the Loop-1 / Loop-2 distinction of §3: Loop 1 fires at the fast timescale, updating a single deployment's scaffold; Loop 2 fires at the slow timescale, consolidating across the deployment's own episodes rather than across deployments. PACE's contribution is to show that the two-loop architecture is implementable under the resource constraints of small, frozen production models — not only under frontier-model assumptions — and that the slow/fast consolidation structure persists even when the optimiser model and the target model are the same frozen small model.

G.9 Synthesis: what the convergence establishes

Reading the full cluster against the §3 mechanism stack and the §9 four-property criterion, four claims are now well-supported by the convergent evidence:

1. **The scaffold-versus-weights decomposition is independently confirmed.** GEPA, Meta-Harness, MOSS, RCL, and PACE all begin from the observation that model weights and the external artifact layer are distinct adaptation surfaces with different cost, reversibility, and scope properties. No surveyed paper disputes this decomposition; the variation is in which substrate each paper targets (skill document, harness code, system prompt, shared memory).
2. **Credit assignment is the central engineering bottleneck.** AHE, CANTANTE, Meta-Harness, and Ong et al. [2026] all identify credit assignment — attributing a system-level failure to a specific scaffold coordinate — as the problem that distinguishes disciplined Loop-1 evolution from trial-and-error. This is §3's operation 3, and it remains unsolved in the general multi-substrate case.
3. **Lifecycle hygiene is a precondition for descent, not an alternative to it.** Ratchet establishes that the gradient-analogue mechanism of §3 requires a maintained scaffold as a precondition. A dirty artifact pool (stale, bloated, miscurated) prevents the gate G_t from receiving a meaningful signal. Hygiene and optimisation are sequential dependencies, not competing paradigms.
4. **No surveyed system combines Loop-1 automation with cross-organisational Loop-2 governance.** SkillOpt has the strongest Loop-1 closure and the most systematic evidence-radius measurement, but it does not implement cross-organisational promotion, versioned lineage with RBAC, or rollback. CORAL implements asynchronous cross-agent promotion within one system but not across organisations. PACE implements two-timescale consolidation within one deployment. Meta-Harness implements Loop-1 closure over the harness substrate but no Loop-2 promotion mechanism. The four-property gap named in §9 is not reduced by any member of this cluster; it is, if anything, confirmed more strongly by the independent convergence of multiple research threads on the same single-organisation ceiling.

The §3 picture is not aspirational. It is the limit that the field is converging on from multiple directions simultaneously, under multiple framings and for multiple substrates. What remains is not the architecture — that exists, in fragments, across SkillOpt, GEPA, Meta-Harness, AHE, MOSS, CANTANTE, Ratchet, PACE, CORAL, and RCL — but the governance layer that would allow a locally-validated edit to be promoted to a cross-organisational scope without losing the auditability, rollback safety, and evidence-radius discipline that make it trustworthy.

G.10 SkillOpt against §9's four properties

Mapped onto §9's four properties of governed cross-tenant scaffold optimization:

- (a) **Auto-gated Loop 1.** *Satisfied.* The held-out selection-split gate at strictly-greater margin instantiates G_t.

- (b) **Multi-tenant cross-org promotion.** *Absent.* SkillOpt is a single-organisation research system; there is no governed cross-tenant aggregator.
- (c) **Versioned lineage with RBAC.** *Partial.* The exported `best_skill.md` is a versioned artifact, and the protected slow-update field separates fast and slow consolidation. There is no documented RBAC layer or audit-trail policy, and `edit_apply_report.json` is an operational log rather than an access-controlled lineage system.
- (d) **Rollback.** *Absent.* The training loop selects the best validation-gated skill and exports it; no rollback envelope on deployed artifacts is reported.

The cross-model, cross-harness, and cross-benchmark transfer experiments reported by SkillOpt are *empirical evidence-radius measurements*: they tell us that a skill optimised in one (model, benchmark, harness) cell continues to improve performance when redeployed in nearby cells without further optimisation. In §3 vocabulary, this is a measured Loop-2 *radius* for the artifact, not a *governance mechanism* for promoting the artifact. Both readings are useful: SkillOpt is the strongest current Loop-1 proof and supplies the first systematic transfer evidence at the scaffold layer, while the §9 cross-tenant governance triad ((b), (c), (d)) remains unfilled.

G.11 Cost and bloat

SkillOpt reports 0.6M to 46.4M training tokens per absolute test-point gain (their Table 6), with the procedural benchmarks (SpreadsheetBench, OfficeQA, LiveMath) at the cheap end and search/visual benchmarks (SearchQA, DocVQA) an order of magnitude more expensive per point. The deployed artifact remains 379–1,995 tokens regardless. This is, to our knowledge, the first published cost-per-point figure for scaffold-layer descent, and it complements the §10 scaffold-bloat discussion (and the Ratchet hygiene-precondition finding walked in G.7): the textual learning rate L_t with cosine decay keeps the deployed skill compact even when the optimiser explores many more edits than are accepted (only 1–4 edits survive into the final skill across the six benchmarks). The mechanism that prevents the deployed artifact from bloating is exactly the complexity penalty $\Omega(z)$ of §3, instantiated as a hard cap on accepted edits per step rather than a soft regulariser.

References

Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S: An open agentic framework that uses computers like a human. *arXiv preprint*, 2024.

Saaket Agashe et al. Agent S2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint*, 2025.

AGENTS.md. AGENTS.md: A cross-vendor open standard for agent instructions, 2025. URL `https://agents.md`.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint*, 2025. Figures cited from the current arXiv version (six tasks, 6% average); an earlier version reported four tasks and a 10% average.

Salaheddin Alzubi et al. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint*, 2026.

Anthropic. Model context protocol specification, 2024. URL `https://modelcontextprotocol.io`.

Mikhail L. Arbuzov, Sisong Bei, Ziwei Dong, Dmitri Kalaev, and Alexey A. Shvets. Beyond exponential decay: Rethinking error accumulation in large language models. *arXiv preprint*, 2025.

Mikhail L. Arbuzov, Alexey A. Shvets, and Sisong Bei. The architecture of errors: Logarithmic mode discovery and polylogarithmic intervention budgets for long-context LLM reliability, 2026. arXiv preprint, forthcoming.

Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. MOSS: Self-evolution through source-level rewriting in autonomous agent systems. *arXiv preprint*, 2026.

Mert Cemri et al. Why do multi-agent LLM systems fail? *arXiv preprint*, 2025. Introduces the MAST taxonomy (Multi-Agent System failure Taxonomy).

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Yixing Chen et al. Multi-agent evolve: LLM self-improve through co-evolution. *arXiv preprint*, 2025.

Diego F. Cuadros, Abdoul-Aziz Maiga, Helen Meskhidze, and Andre Curtis-Trudel. Governed collaborative memory as artificial selection in LLM-based multi-agent systems. *arXiv preprint*, 2026.

Shibhansh Dohare et al. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. doi: 10.1038/s41586-024-07711-7.

Alhussein Fawzi, Matej Balog, Aja Huang, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. DeepMind AlphaTensor.

Harvey. Harvey product overview, 2026. URL `https://www.harvey.ai`.

Yufei He et al. EvoTest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint*, 2025.

Hippocratic AI. Polaris clinical outcome evidence, 2026. URL `https://www.hippocraticai.com`.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems (ADAS). *arXiv preprint*, 2024.

Xu Huang et al. CASCADE: Cumulative agentic skill creation through autonomous development and evolution. *arXiv preprint*, 2025.

Daniel Jaroslawicz et al. How many instructions can LLMs follow at once? *arXiv preprint*, 2025. Introduces the IFScale benchmark.

Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, et al. Understanding the effects of RLHF on LLM generalisation and diversity. In *ICLR 2024*, 2024.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint*, 2026.

Hanchen Li, Runyuan He, Qizheng Zhang, Changxiu Ji, Qiuyang Mang, Xiaokun Chen, Lakshya A. Agrawal, Wei-Liang Liao, Eric Yang, Alvin Cheung, James Zou, Kunle Olukotun, Ion Stoica, and Joseph E. Gonzalez. Combee: Scaling prompt learning for self-improving language model agents. *arXiv preprint*, 2026.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint*, 2026.

Chen Ling, Pei Chen, Albert Guan, Jiaming Qu, Shayan Ali Akbar, Madhu Gopinathan, and Erwin Cornejo. PACE: Two-timescale self-evolution for small language model agents. *arXiv preprint*, 2026.

Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2023.

Xingyan Liu et al. SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint*, 2026. Accepted at ACM SIGIR 2026 Industry Track.

Jai Lal Lulla et al. On the impact of AGENTS.md files on the efficiency of AI coding agents. *arXiv preprint*, 2026.

Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding Plasticity in Neural Networks. In *International Conference on Machine Learning (ICML)*, 2023. Oral presentation. The earlier eprint 2303.07507 in the source References list was a different paper (Abbas et al., RL plasticity).

Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Jingwei Ni et al. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint*, 2026.

Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025.

Kai Tzu-iunn Ong, Minseok Kang, Dongwook Choi, Junhee Cho, Seungju Kim, Seungwon Lim, Geunha Jang, Minwoo Oh, Bogyung Jeong, Sunghwan Kim, Taeyoon Kwon, and Jinyoung Yeo. Towards direct evaluation of harness optimizers via priority ranking. *arXiv preprint*, 2026.

Vishakh Padmakumar and He He. Does writing with language models reduce content diversity? *arXiv preprint*, 2023.

Joon Sung Park et al. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint*, 2026.

Yuval Shalev, Zifeng Ding, and Mateja Jamnik. Training language agents to learn from experience. *arXiv preprint*, 2026.

Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Sierra. Sierra agent os, 2024. URL `https://sierra.ai`.

Sierra. Agent OS 2.0: From answers to memory and action, 2025. URL `https://sierra.ai/blog/agent-os-2-0`.

Shivalika Singh et al. The leaderboard illusion. *arXiv preprint*, 2025. Audit references NeurIPS 2025 venue; not confirmed on the arXiv page. Verify before journal citation.

Weijia Song, Jiashu Yue, and Zhe Pang. ABSTRAL: Automatic design of multi-agent systems through iterative refinement and topology optimization. *arXiv preprint*, 2026.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents (CoALA). *Transactions on Machine Learning Research (TMLR)*, 2023.

Weihao Tan et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint*, 2024.

Md Mehrab Tanjim, Jayakumar Subramanian, Xiang Chen, Branislav Kveton, Subhojyoti Mukherjee, Anlan Zhang, Sungchul Kim, Somdeb Sarkhel, and Sunav Choudhury. MOCHA: Multi-objective chebyshev annealing for agent skill optimization. *arXiv preprint*, 2026.

Rishabh Tiwari, Kusha Sareen, Lakshya A. Agrawal, Joseph E. Gonzalez, Matei Zaharia, Kurt Keutzer, Inderjit S. Dhillon, Rishabh Agarwal, and Devvrit Khatri. Learning, fast and slow: Towards LLMs that adapt continually. *arXiv preprint*, 2026.

Dat Tran and Douwe Kiela. Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv preprint*, 2026. Replaces the previous (hallucinated) "Memento-Skills" entry that mis-attributed this arXiv ID. The actual paper at this ID is a contrast finding: single-agent baselines match or beat multi-agent systems under matched thinking-token budgets.

Nikita Vassilyev, William Berrios, Ruowang Zhang, Bo Han, Douwe Kiela, and Shikib Mehri. Reflective context learning: Studying the optimization primitives of context space. *arXiv preprint*, 2026.

Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv preprint*, 2023.

Jiongxiao Wang et al. Reinforcement learning for self-improving agent with skill library. *arXiv preprint*, 2025a. Method is named SAGE (Skill Augmented GRPO for self-Evolution); not in the title.

Xiaoxing Wang et al. AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents. *arXiv preprint*, 2026.

Zhenhailong Wang et al. Mobile-Agent-E: Self-evolving mobile assistant for complex tasks. *arXiv preprint*, 2025b.

Rong Wu et al. EvolveR: Self-evolving LLM agents through an experience-driven lifecycle. *arXiv preprint*, 2025. Accepted at ICML 2026.

Xiyang Wu et al. Co-evolving LLM decision and skill bank agents for long-horizon tasks. *arXiv preprint*, 2026. Framework name is COSPLAY; does not appear in the title.

Zhiyong Wu et al. OS-Copilot: Towards generalist computer agents with self-improvement. *arXiv preprint*, 2024.

Peng Xia et al. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint*, 2026.

Jinhang Xu et al. NanoResearch: Co-evolving skills, memory, and policy for personalized research automation. *arXiv preprint*, 2026a.

Zhenlin Xu et al. From storage to steering: Memory control flow attacks on LLM agents. *arXiv preprint*, 2026b. Introduces the MCFA attack class and the MEMFLOW evaluation framework.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills. arXiv preprint arXiv:2605.23904, 2026. URL `https://microsoft.github.io/SkillOpt`. Microsoft, Shanghai Jiao Tong University, Tongji University, Fudan University. Beats TextGrad, GEPA, Trace2Skill, and EvoSkill baselines on 52/52 evaluated cells.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint*, 2024.

Tom Zehle. CANTANTE: Optimizing agentic systems via contrastive credit attribution. *arXiv preprint*, 2026.

Hanrong Zhang et al. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint*, 2026a.

Jenny Zhang et al. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint*, 2025a. Referenced as DGM in body text.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, et al. Agentic context engineering: Evolving contexts for self-improving language models. arXiv preprint arXiv:2510.04618, 2025b.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Ratchet: A minimal hygiene recipe for self-evolving LLM agents. *arXiv preprint*, 2026b.

Yingli Zhou, Wang Shu, Yaodong Su, et al. A comprehensive survey on agent skills: Taxonomy, techniques, and applications. *arXiv preprint*, 2026.

Wei Zou et al. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint*, 2024. Accepted at USENIX Security 2025.